

دليل مشروع ShopApp — شرح تفصيلي سطر بسطر

دليل تعليمي كامل لمشروع **ASP.NET Core MVC 8** — من أول سطر في **Program.cs** لحد آخر Tag Helper في الـ Views.

إزاي تستخدم الدليل ده

الدليل مقسوم لـ ١٦ فصل. كل فصل بيشرح جزء من المشروع، وبينتهي بـ **تمارين عملية** تحت عنوان "جرب بنفسك".

نصيحة مهمة: مفتوح عندك الـ VS Code جنب الدليل ده. كل ما تقرا شرح ملف، افتح الملف نفسه وشوف الكلام على أرض الواقع. القراءة لوحدها مش هتثبت المعلومة — التجربة هي اللي بتثبتها.

ترتيب القراءة المقترح للمبتدى:

الترتيب	الفصل	الموضوع
١	الفصل 1	إيه هو MVC أصلاً؟ — المفهوم
٢	الفصل 3	رحلة الـ Request — الصورة الكبيرة
٣	الفصل 5	الموديلات — البيانات
٤	الفصل 7	العلاقات — الربط
٥	الفصل 8	الـ DbContext — قاعدة البيانات
٦	الفصل 10	Dependency Injection — أهم مفهوم في .NET. الحديث
٧	الفصل 12	Program.cs — نقطة التجميع
٨	الفصل 13	الـ Controllers — المنطق
٩	الفصل 15	الـ Views — الواجهة
١٠	الفصل 16	Migrations — إدارة الداتايز

باقي الفصول مرجعية — ارجعها وقت الحاجة.

#	الفصل	بيجاب على سؤال
1	إيه هو ASP.NET Core MVC؟	ليه أصلاً بنقسم المشروع كده؟
2	خريطة المشروع	كل ملف بيعمل إيه؟
3	رحلة ال Request	إيه اللي بيحصل لما أكتب URL؟
4	ملف <code>.csproj</code>	إزاي المشروع بيعرف الباكجات؟
5	<code>Models/Category.cs</code>	إزاي أعرف بيانات؟
6	<code>Models/Product.cs</code>	إيه ال Data Annotations دي؟
7	العلاقات (Relations)	إزاي أربط جدولين؟
8	<code>Data/AppDbContext.cs</code>	إزاي الكود بيكلم الداتابيز؟
9	<code>appsettings.json</code>	فين يحط الإعدادات؟
10	Dependency Injection	إيه ده وليه مهم؟
11	<code>Services/</code>	ليه أعمل طبقة زيادة؟
12	<code>Program.cs</code>	إزاي كل ده بيتجمع؟
13	<code>CategoriesController.cs</code>	إزاي أعمل CRUD؟
14	<code>ProductsController.cs</code>	إيه الفرق بين النمطين؟
15	ال Views و Razor	إزاي أعرض البيانات؟
16	Migrations	إزاي أعدّل الداتابيز؟
17	الأمان	إيه الحماية المطبقة؟
18	أخطاء شائعة	لما حاجة تبوط أعمل إيه؟
19	خطواتك الجاية	أتعلم إيه بعد كده؟

الفصل 1: إيه هو ASP.NET Core MVC؟

المشكلة اللي MVC بيحلها

تخيل إنك عايز تعمل صفحة تعرض منتجات. من غير أي تنظيم، هتكتب ملف واحد فيه:

- كود بيتصل بالداتابيز
- كود بيتحقق إن السعر مش سالب
- كود HTML بيرسم الجدول
- كود بيتعامل مع زرار "احذف"

الملف ده هيبقى ٥٠٠ سطر. ولما تيجي تغيّر شكل الجدول، هتلف وسط كود الداتابيز. ولما تيجي تختبر التحقق من السعر، مش هتقدر — لأنه متشابه مع ال HTML.

MVC بيقسّم المسؤولية على ثلاث أجزاء:

الجزء	مسؤوليته	في مشروعنا
Model	شكل البيانات وقواعدها	Category.cs و Product.cs
View	العرض والشكل فقط	ملفات *.cshtml
Controller	استقبال الطلب وتنسيق الرد	ProductsController.cs

القاعدة الذهبية

ال Controller مش بيرسم HTML، وال View مش بتكلم الداتابيز.

كل واحد في حته. لما تحب تغيّر الشكل → تفتح ال View بس. لما تحب تغيّر منطق الحفظ → تفتح ال Controller أو ال Service بس.

فين ال Model في مشروعنا بالضبط؟

كلمة "Model" في MVC أوسع من مجرد كلاس البيانات. عندنا:

الـمـلف	دوره في الـ "M"
<code>Models/Product.cs</code>	الـ Entity — شكل البيانات وقواعد التحقق
<code>Data/AppDbContext.cs</code>	طبقة الوصول للـ داتا بيز
<code>Services/ProductService.cs</code>	منطق العمل (Business Logic)

الـ ثلاثة دول مجتمعين بيشكلوا الـ "M" في MVC.

جرب بنفسك — الفصل 1

١. افتح `Views/Products/Index.cshtml` ودور على أي سطر بيتصل بالداتا بيز. مش هتلاقي — ده الفصل بين المسؤوليات.

٢. افتح `Controllers/ProductsController.cs` ودور على أي وسم HTML. مش هتلاقي كمان.

٣. سؤال للتفكير: لو عايز تغير ألوان جدول المنتجات، هتفتح أنهي ملف؟ ولو عايز تمنع حفظ منتج سعره أقل من ١٠٠ جنيه؟

الإجابة

- تغيير الألوان → `Views/Products/Index.cshtml` (فقط View)
 - منع سعر أقل من ١٠٠ → في الـ `Models/Product.cs` [Range] (فقط Model)
- ولاحظ إن كل تعديل في ملف واحد بس. ده بالضبط اللي MVC اتعمل عشانه.

الفصل 2: خريطة المشروع

الشجرة الكاملة

```
ShopApp/
├── Models/
│   ├── Category.cs      ← طبقة البيانات
│   ├── Product.cs       ← الموديل الأول - التصنيف
│   ├── Product.cs       ← الموديل الثاني - المنتج
│   └── ErrorViewModel.cs ← موديل صفحة الخطأ - جاي مع القالب
├── Data/
│   └── ApplicationDbContext.cs ← طبقة الوصول للبيانات
│                                   ← العلاقة + البيانات المبدئية + DbContext الى
├── Services/
│   ├── IProductService.cs ← طبقة منطق العمل
│   └── ProductService.cs  ← (Interface) العقد
│                                   ← التنفيذ
├── Controllers/
│   ├── CategoriesController.cs ← طبقة استقبال الطلبات
│   ├── ProductsController.cs  ← التصنيفات CRUD
│   └── HomeController.cs      ← المنتجات CRUD
│                                   ← التحويل + صفحة الخطأ
├── Views/
│   ├── Categories/
│   │   ├── Index.cshtml      ← طبقة العرض
│   │   ├── Create.cshtml     ← قائمة التصنيفات
│   │   ├── Edit.cshtml       ← فورم الإضافة
│   │   ├── Details.cshtml    ← فورم التعديل
│   │   └── Delete.cshtml      ← صفحة التفاصيل
│   ├── Products/
│   │   └── Delete.cshtml      ← تأكيد الحذف
│   ├── Shared/
│   │   ├── _Layout.cshtml     ← نفس الخمس صفحات
│   │   ├── _Layout.cshtml     ← قالب المشترك - الهيدر والفوتر
│   │   ├── Error.cshtml       ← صفحة الخطأ
│   │   └── _ValidationScriptsPartial.cshtml ← سكريبتات التحقق
│   ├── _ViewImports.cshtml    ← Views لكل الـ Tag Helpers والـ using الـ
│   └── _ViewStart.cshtml      ← الافتراضي Layout يحدد الـ
├── Migrations/
│   ├── 20260819084454_InitialCreate.cs      ← تاريخ تغييرات الداتابيز
│   ├── 20260819084454_InitialCreate.Designer.cs ← Up و Down - الأوامر
│   └── ApplicationDbContextModelSnapshot.cs ← لحظة وقت الإنشاء
│                                   ← اللقطة الحالية
├── wwwroot/
│   ├── css/site.css            ← الملفات الثابتة - يوصلها المتصفح مباشرة
│   ├── js/site.js
│   └── lib/
│       ├── Bootstrap و jQuery
```

Properties/	
└─ launchSettings.json	إعدادات التشغيل المحلي - البورت والبيئة ←
Program.cs	نقطة البداية ←
appsettings.json	Connection String - الإعدادات ←
appsettings.Development.json	إعدادات تطوير - بتطغى على اللي فوق ←
ShopApp.csproj	تعريف المشروع والباكدجات ←
shopapp.db	بيتولد تلقائياً - SQLite ملف قاعدة بيانات ←

القواعد اللي ASP.NET بيعتمد عليها (Conventions)

ASP.NET Core بيعتمد على الاصطلاح بدل الإعداد — يعني بيّفهم نيتك من التسمية من غير ما تكتب إعدادات:

الاصطلاح	المثال	النتيجة
اسم الكنترولر لازم ينتهي بـ Controller	ProductsController	الراوت بيبقى /Products
ال View لازم تكون في Views/<اسم الكنترولر>/	Views/Products/	return View() بيلاقها لوحده
اسم ملف ال View = اسم ال Action	Index() → Index.cshtml	ربط تلقائي
ملف يبدأ بـ _ = ملف مساعد	_Layout.cshtml	مش صفحة مستقلة
اسم Id أو <Class>Id	Id في Product	Primary Key تلقائي
اسم <Navigation>Id	CategoryId	Foreign Key تلقائي

ليه ده مهم؟ لأنك لما تخالف الاصطلاح، الحاجات بتقف عن الشغل من غير رسالة خطأ واضحة. لو حطيت **Index.cshtml** في **Views/Product/** (مفرد) بدل **Views/Products/**، هتاخد خطأ "View not found".

جرب بنفسك — الفصل 2

١. غير اسم فولدر **Views/Products** لـ **Views/Product** وشغل المشروع. هتاخد إيه؟ رجّعه تاني.

٢. افتح `Views/_ViewStart.cshtml` — سطر واحد بس. اقرا محتواه وفكر: ده بيعمل إيه؟

الإجابة

```
@{
    Layout = "_Layout";
}
```

بيقول لكل الـ Views في المشروع: "استخدم `_Layout.cshtml` كقالب". عشان كده مش محتاج تكتب السطر ده في كل View لوحدها. لو عايز View معينة من غير Layout، اكتب فيها `@{ Layout = null; }`.

٣. فولدر `wwwroot` — جرب تفتح `http://localhost:5124/css/site.css` في المتصفح. هيفتح عادي. دلوقتي جرب `http://localhost:5124/Program.cs`. مش هيفتح — ليه؟

الإجابة

`wwwroot` هو الفولدر الوحيد المكشوف للمتصفح (عن طريق `app.UseStaticFiles()`). أي ملف بره الفولدر ده — كودك، إعداداتك، الداتا بيز — محمي ومش قابل للتحميل. ده إجراء أمني أساسي.

الفصل 3: رحلة ال Request

ده أهم فصل في الدليل. لو فهمت الرحلة دي، هتفهم ASP.NET كله.

السيناريو: المستخدم كتب `localhost:5124/Products/Details/2`

الخطوات التسعة

١ — المتصفح يبعث الطلب

```
GET /Products/Details/2 HTTP/1.1
Host: localhost:5124
```

٢ — **Kestrel** يستقبله سيرفر الويب المدمج في .NET. بيستلم الطلب ويحوّله لكائن `HttpContext`.

٣ — الطلب يمر على ال **Middleware Pipeline** بالترتيب اللي في `Program.cs`:

الترتيب	ال Middleware	بيعمل إيه
١	<code>UseHttpsRedirection()</code>	لو الطلب على http يحوّله ل https
٢	<code>UseStaticFiles()</code>	لو الطلب لملف في <code>wwwroot</code> بيعته ويوقف هنا
٣	<code>UseRouting()</code>	يحلّل ال URL ويحدد الوجهة
٤	<code>UseAuthorization()</code>	يفحص الصلاحيات (مفيش حماية في مشروعنا دلوقتي)

٤ — ال **Router** يطابق ال URL مع القالب `{controller=Products}/{action=Index}/{id?}`:

الجزء في ال URL	المتغير	القيمة
Products	controller	Products
Details	action	Details
2	id	2

٥ — ال DI Container يبني الكترولر بسلسلة اعتماديات:

المطلوب	يحتاج	النتيجة
ProductsController	IProductService	يعمل ProductService
ProductService	AppDbContext	يعمل AppDbContext
AppDbContext	DbContextOptions	ياخذها من الإعدادات

وبعدها: `new ProductsController(productService, context)`

٦ — **Model Binding** يحوّل "2" النصية لـ `int` ويحطها في المعطى، ثم ينادي `await controller.Details(2)`

٧ — ال **Action** ينفذ `await _productService.GetByIdAsync(2)` و EF Core يولّد:

```
SELECT p.*, c.* FROM Products p
LEFT JOIN Categories c ON p.CategoryId = c.Id
WHERE p.Id = 2
```

والنتيجة كائن `Product` جواه `Category`.

٨ — `return View(product)` — Razor يدور على `Views/Products/Details.cshtml`، يلقها في `Views/Shared/_Layout.cshtml`، ويحوّل الكل لـ HTML.

٩ — الرد يرجع عكس ال Pipeline لحد المتصفح. وبعدها ال `DbContext` يتقفل ويتخلص منه (لأنه `Scoped`).

نقطة مهمة: ال Middleware زي طبقات البصلة

ال Middleware مش مجرد قائمة بتننفذ بالترتيب — كل واحد بيلف اللي بعده:

Request → [HttpsRedirection [StaticFiles [Routing [Endpoint]]]] → Response

يعني الطلب بيدخل من برّه لجوّه، والرد بيخرج من جوّه لبرّه.

يعني كل Middleware بيشفو الطلب وهو داخل والرد وهو خارج. وده السبب إن الترتيب مهم جداً.

مثال على خطأ الترتيب: لو حطيت `UseAuthorization()` قبل `UseRouting()`، نظام الصلاحيات مش هيعرف المستخدم رايح على أنهي صفحة، فمش هيعرف يقرر يسمحله ولا لأ.

جرب بنفسك — الفصل 3

١. في `Program.cs`، انقل سطر `app.UseStaticFiles();` وحطه بعد `app.UseRouting();`. شغل وشوف لو الـ CSS لسه شغال. (هيشغل، بس بكفاءة أقل — الطلب بيمر على الـ Router الأول من غير داعي).

٢. جرب تفتح `localhost:5124/Products/Details/999`. إيه اللي بيحصل؟ وليه؟

الإجابة

بترجعك صفحة 404. لأن في الـ Action:

```
var product = await _productService.GetByIdAsync(id.Value);  
if (product is null) return NotFound();
```

الـ `GetByIdAsync` رجّع `null` لأن مفيش منتج بالـ Id ده، فالـ Action رجّع `NotFound()` اللي بتترجم لـ HTTP 404.

٣. جرب تفتح `localhost:5124/Products` من غير action. ليه اشتغلت ورجعت القائمة؟

الإجابة

بسبب القيمة الافتراضية في قالب الراوت:

```
pattern: "{controller=Products}/{action=Index}/{id?}"
```

`action=Index` معناها: لو المستخدم مكتبش action، استخدم `Index`. وعلامة `?` في `{id?}` معناها إن الـ id اختياري.

٤. جرب تفتح `localhost:5124` لوحده. هيوديك على `/Products`. دور في `HomeController.cs` على السبب.

الفصل 4: ملف ShopApp.csproj

ده ملف تعريف المشروع. ال `dotnet` CLI بيقراه عشان يعرف يبني إيه وبأي أدوات.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.11" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.11">
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

</Project>
```

سطر بسطر

بيقول إن ده مشروع ويب. ال SDK ده بيجيب معاه تلقائياً كل مكتبات ASP.NET Core (`Kestrel`, `Routing`, `Razor`...) من غير ما تضيفها يدوي. لو كان مشروع Console كان هيبقى `Microsoft.NET.Sdk`.

إصدار .NET المستهدف. لازم يكون عندك SDK الإصدار ده أو أحدث. `<TargetFramework>net8.0</TargetFramework>`

دي ميزة قوية جداً في #C الحديثة اسمها **Nullable Reference Types**. لما تتفعل: `<Nullable>enable</Nullable>`

```
string name;    // أبدأً null الكومبايلر يفترض إنها مش هتبقى ←
string? name;   // null علامة ؟ معناها "ممكن تبقى ←"
```

الكومبايلر بيحذرك لو حاولت تستخدم متغير ممكن يكون `null` من غير ما تتأكد. ده بيمنع أشهر خطأ في البرمجة:

`NullReferenceException`.

في مشروعنا شوف الفرق:

```
public string Name { get; set; } = string.Empty; // مطلوب، فبنديله قيمة ابتدائية
public string? Description { get; set; } // اختياري، ممكن يفضل null
public Category? Category { get; set; } // ممكن ما تتحطّش من الداتايز
```

، `System.Linq` ، `System`) تلقائي لأشهر المكتبات `using` يضيف `<ImplicitUsings>enable</ImplicitUsings>` في أول أي ملف — هي موجودة ضمناً.

`<PrivateAssets>all</PrivateAssets>` موجودة على باكج `EntityFrameworkCore.Design` تحديداً. معناها: "الباكج ده أداة وقت التطوير بس، متضمّهوش في النسخة النهائية اللي هترفعها على السيرفر". الباكج ده بيستخدمه `dotnet ef` عشان يعمل الـ Migrations — التطبيق نفسه وهو شغال مش محتاجه.

جرب بنفسك — الفصل 4

١. شيل `<Nullable>enable</Nullable>` وشغل `dotnet build`. هتلاحظ إن التحذيرات اختفت — بس كمان الحماية اختفت. رجّعها.

٢. جرب تضيف باكج جديد وشوف الملف بيتغير إزاي:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

افتح الـ `.csproj`. بعدها. بعد كده شيله:

```
dotnet remove package Microsoft.EntityFrameworkCore.SqlServer
```

الكود الكامل

```
using System.ComponentModel.DataAnnotations;

namespace ShopApp.Models;

public class Category
{
    public int Id { get; set; }

    [Required(ErrorMessage = "اسم التصنيف مطلوب")]
    [StringLength(60, MinimumLength = 2, ErrorMessage = "الاسم لازم يكون بين 2 و 60 حرف")]
    [Display(Name = "اسم التصنيف")]
    public string Name { get; set; } = string.Empty;

    [StringLength(250)]
    [Display(Name = "الوصف")]
    public string? Description { get; set; }

    [Display(Name = "تاريخ الإنشاء")]
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public ICollection<Product> Products { get; set; } = new List<Product>();
}
```

سطر بسطر

namespace ShopApp.Models;

لاحظ الفاصلة المنقوطة في الآخر من غير أقواس `{ }`. دي ميزة C# 10 اسمها **File-scoped namespace** — كل الملف جوه ال namespace ده. بتوفّر مستوى إزاحة كامل مقارنة بالطريقة القديمة.

public int Id { get; set; }

المفتاح الأساسي (Primary Key). EF Core بيتعرف عليه أوتوماتيك بقاعدتين:

1. خاصية اسمها **Id** بالظبط

2. أو خاصية اسمها `Id` <اسم الكلاس> — يعني `CategoryId` هنا برضه كان هيشغل

ولأن نوعه `int` ، EF بيخليه **AUTOINCREMENT** تلقائياً. شوف SQL اللي اتولد:

```
"Id" INTEGER NOT NULL CONSTRAINT "PK_Categories" PRIMARY KEY AUTOINCREMENT
```

لو حبيت تسميه اسم ثاني خالص (مثلاً `CategoryCode`)، لازم تقوله صراحة:

```
[Key]
public int CategoryCode { get; set; }
```

`[Required(ErrorMessage = "...")]`

ده **Data Annotation** — سمة (Attribute) بتضيف معلومة إضافية على الخاصية. `[Required]` بتعمل حاجتين في نفس الوقت:

المكان	التأثير
في الداتابيز	العمود بيبقى <code>NOT NULL</code>
في الفورم	لو المستخدم ساب الحقل فاضي، <code>ModelState.IsValid</code> بترجع <code>false</code> والرسالة بتظهر

والرسالة بتظهر في ال View من خلال:

```
<span asp-validation-for="Name" class="text-danger"></span>
```

`[StringLength(60, MinimumLength = 2)]`

بتحدد أقصى وأدنى طول. تأثيرها: - في SQL Server: العمود بيبقى `NVARCHAR(60)` بدل `NVARCHAR(MAX)` - في التحقق: نص أقل من حرفين أو أكثر من ٦٠ بيفرض

ملحوظة: في SQLite كل النصوص نوعها `TEXT` من غير حد أقصى، فالقيد بيتطبق في مستوى التحقق بس مش في الداتابيز. لو حولت ل SQL Server هتلاقي الفرق.

`[Display(Name = "الترتيب")]`

بتحدد الاسم اللي هيفظهر للمستخدم. بستخدم في:


```
<label asp-for="Name"></label> <!-- هيطع: اسم التصنيف -->
@Html.DisplayNameFor(m => m.Name) <!-- نفس النتيجة -->
```

من غيرها كان هيطع اسم الخاصية بالإنجليزي: **Name** . دي طريقة نظيفة تخلي الواجهة عربي والكود إنجليزي.

```
public string Name { get; set; } = string.Empty;
```

ليه **string.Empty** ؟ بسبب **<Nullable>enable</Nullable>** . الكومبايلر عايز يتأكد إن أي خاصية **string** (مش **string?**) عندها قيمة من أول لحظة. من غير القيمة الابتدائية دي هتأخذ تحذير:

```
Warning CS8618: Non-nullable property 'Name' must contain a non-null value
when exiting constructor.
```

```
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

ليه **UtcNow** مش **Now** ؟

DateTime.Now بيرجع وقت الجهاز اللي شغال عليه السيرفر. لو السيرفر في أمريكا والمستخدم في مصر، الأرقام هتبقى مختلفة. ولو نقلت السيرفر لمنطقة زمنية ثانية، البيانات القديمة هتبقى غلط.

القاعدة: خزن دائماً بـ UTC، واعرض بالتوقيت المحلي وقت العرض:

```
// وقت التخزين
CreatedAt = DateTime.UtcNow;

// وقت العرض
@Model.CreatedAt.ToLocalTime().ToString("yyyy-MM-dd HH:mm")
```

```
public ICollection<Product> Products { get; set; } = new List<Product>();
```

دي **Navigation Property** — أهم سطر في الملف من ناحية العلاقات. الفصل السابع كله عنها.

ليه **ICollection** مش **List** ؟ لأن EF Core محتاج يستبدلها بمجموعة خاصة بيه (Proxy) عشان يتتبع التغييرات. **ICollection** هي أبسط واجهة بتديله المرونة دي. **List** كانت هتشتغل بس بتقيده.

ليه القيمة الابتدائية **= new List<Product>()** ؟ عشان تتجنب **NullReferenceException** . لو التصنيف لسه اتعمل ومفيهوش منتجات، **category.Products.Count** هترجع 0 بدل ما ترمي استثناء.

جرب بنفسك — الفصل 5

١. غيّر `[StringLength(60, MinimumLength = 2)]` لـ `MinimumLength = 5`، شغل، وحاول تضيف تصنيف اسمه "أب". شوف الرسالة بتظهر فين وإزاي.

٢. شيل `[Display(Name = "اسم التصنيف")]` وشوف الليبل في صفحة الإضافة بقي إيه.

٣. أضف خاصية جديدة:

```
[Display(Name = "مفعّل")]
public bool IsActive { get; set; } = true;
```

شغل المشروع دلوقتي. هيقع. ليه؟

الإجابة

هتأخذ استثناء زي:

```
SQLite Error 1: 'no such column: c.IsActive'
```

لأنك غيرت الـ **Model** بس منقلتش للـ **Database**. الكود دلوقتي بيدور على عمود مش موجود.

الحل: اعمل Migration جديدة تنقل التغيير:

```
dotnet ef migrations add AddIsActiveToCategory
dotnet ef database update
```

الدرس المهم: أي تعديل في الموديلات لازم يتبعه **Migration**. الفصل ١٦ بيشرح ده بالتفصيل.

الكود الكامل

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace ShopApp.Models;

public class Product
{
    public int Id { get; set; }

    [Required(ErrorMessage = "اسم المنتج مطلوب")]
    [StringLength(100)]
    [Display(Name = "اسم المنتج")]
    public string Name { get; set; } = string.Empty;

    [Range(0.01, 1_000_000, ErrorMessage = "السعر لازم يكون أكبر من صفر")]
    [Column(TypeName = "decimal(18,2)")]
    [Display(Name = "السعر")]
    public decimal Price { get; set; }

    [Range(0, int.MaxValue, ErrorMessage = "الكمية لا يمكن أن تكون سالبة")]
    [Display(Name = "الكمية بالمخزن")]
    public int Stock { get; set; }

    [Display(Name = "متاح للبيع")]
    public bool IsAvailable { get; set; } = true;

    [Required(ErrorMessage = "لازم تختار تصنيف")]
    [Display(Name = "التصنيف")]
    public int CategoryId { get; set; }

    public Category? Category { get; set; }
}
```

النقاط الجديدة

```
public decimal Price { get; set; }
```

قاعدة ذهبية: استخدم **decimal** للفلوس، مش **double** ولا **float**.

ليه؟ لأن `double` بيخزن الأرقام بنظام ثنائي (Binary Floating Point)، وده بييسبب أخطاء تقريب:

```
double a = 0.1 + 0.2;
Console.WriteLine(a);           // 0.30000000000000004
Console.WriteLine(a == 0.3);    // False ← كارثة في نظام محاسبي

decimal b = 0.1m + 0.2m;
Console.WriteLine(b);           // 0.3
Console.WriteLine(b == 0.3m);   // True ← صح
```

`decimal` بيخزن بنظام عشري، فدقته مضمونة للفلوس. أبطأ شوية لكن الدقة أهم.

`[Column(TypeName = "decimal(18,2)")]`

بتحدد نوع العمود في الداتابيز بالظبط: `18` = إجمالي عدد الأرقام (الدقة) - `2` = عدد الأرقام بعد العلامة العشرية

يعني بتقدر تخزن لحد `9,999,999,999,999.99`.

ليه محتاجينها؟ لأن EF من غيرها في SQL Server بيستخدم `decimal(18,2)` كافتراضي بس بيطلع تحذير. وفي حالات تانية ممكن يقرب الأرقام من غير ما تحس. تحديدها صراحةً بيوضح النية ويمنع المفاجآت.

`[Range(0.01, 1_000_000)]`

الشرطة السفلية في `1_000_000` هي **Digit Separator** — ميزة في C# لتسهيل القراءة. الكومبايلر بيتجاهلها تماماً. `1_000_000` و `1000000` نفس الشيء بالظبط.

`[Range(0, int.MaxValue)]`

بتمنع الكميات السالبة. `int.MaxValue` = `2,147,483,647`.

`public int CategoryId { get; set; }`

المفتاح الأجنبي (Foreign Key). EF Core بيتعرف عليه بالاصطلاح:

اسم الخاصية = اسم (navigation) `Category` + `Id` + Navigation Property + `Id` `CategoryId` ✓

وده اللي خلى EF يولد:

```
CONSTRAINT "FK_Products_Categories_CategoryId"
FOREIGN KEY ("CategoryId") REFERENCES "Categories" ("Id") ON DELETE CASCADE
```

```
public Category? Category { get; set; }
```

Navigation Property للطرف الواحد. من خلالها بتوصل لبيانات التصنيف مباشرة:

```
product.Category.Name // "معالجات"
```

ليه **Category?** بعلامة استفهام؟ لأنها مش دائماً محمّلة. لو جبت المنتج كده:

```
var p = await _context.Products.FindAsync(1);
Console.WriteLine(p.Category.Name); // ❗ NullReferenceException
```

Category هتبقى **null** لأنك ما طلبتش من EF يجيبها. لازم تستخدم **Include**:

```
var p = await _context.Products
    .Include(x => x.Category) // ← JOIN دلوقتي هيعمل
    .FirstOrDefaultAsync(x => x.Id == 1);

Console.WriteLine(p.Category.Name); // ✓ "معالجات"
```

ده بالظبط سبب علامة الاستفهام — الكومبايلر بيفكر إننا ممكن تكون فاضية، فتستخدم **?.** بدل **.**:

```
@item.Category?.Name // هيطبع فاضي بدل ما يقع null آمن - لو
```

جرب بنفسك — الفصل 6

١. في `ProductService.GetAllAsync()`، شيل سطر `.Include(p => p.Category)` وشغل. افتح `/Products` — إيه اللي اختفى من الجدول؟ وليه الصفحة ما وقعتش؟

الإجابة

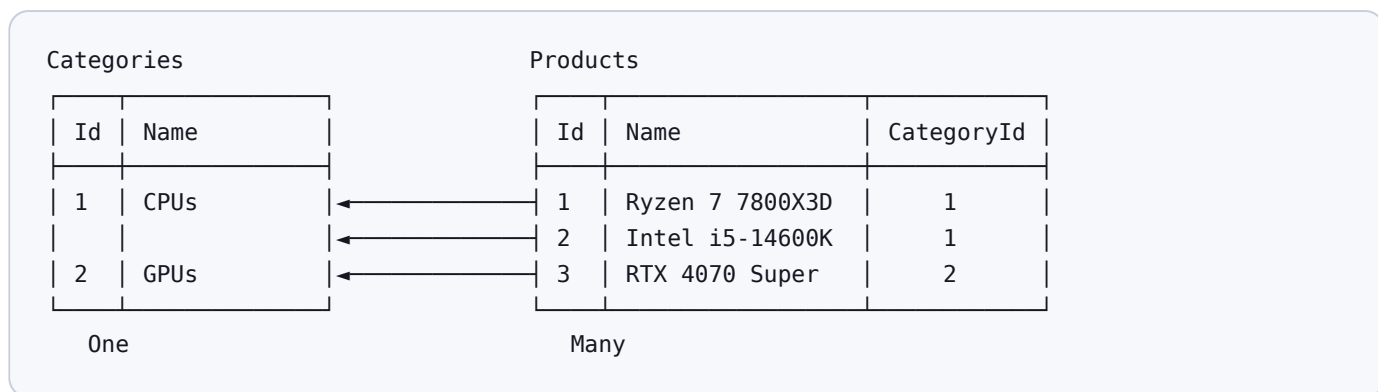
عمود التصنيف هيبقى فاضي. الصفحة ما وقعتش لأن ال `View` بتستخدم `@item.Category?.Name` بعلامة الاستفهام — لو كانت مكتوبة `@item.Category.Name` من غير **?** كانت هتقع بـ `NullReferenceException`.
ده بالظبط سبب وجود **?** في تعريف الخاصية.

٢. غيّر نوع `Price` من `decimal` لـ `double` واعمل Migration. بعدها ضيف منتج بسعر `0.1` وتاني بـ `0.2` ، واعمل مجموعهم في كود. جرّب نفس الحاجة بـ `decimal` . لاحظ الفرق.
٣. غيّر `[Range(0.01, 1_000_000)]` لـ `[Range(100, 5000)]` وحاول تضيف منتج بسعر ٥٠. اقرأ الرسالة اللي هتظهر — من فين جت؟
-

الفصل 7: العلاقات (Relations)

الفكرة الأساسية

في قواعد البيانات العلاقية، الجداول بتترابط بـ **المفاتيح**. الجدول التابع بيحتفظ بمفتاح الجدول الأصلي.



الجدول الشمال (**Categories**) هو الطرف **One**، والجدول اليمين (**Products**) هو الطرف **Many**.

التصنيف رقم ١ ليه منتجين. التصنيف رقم ٢ ليه منتج واحد. دي علاقة **One-to-Many**.

الأركان الأربعة لأي علاقة في EF Core

#	الركن	في الكود	في الداتابيز
١	Primary Key في الأصلي	<code>Category.Id</code>	<code>PK_Categories</code>
٢	Foreign Key في التابع	<code>Product.CategoryId</code>	عمود <code>CategoryId</code>
٣	Navigation من التابع للأصلي	<code>Product.Category</code>	(مش موجود — مفهوم في الكود بس)
٤	Navigation من الأصلي للتابع	<code>Category.Products</code>	(مش موجود — مفهوم في الكود بس)

نقطة مهمة جداً: الأركان ٣ و ٤ مش موجودة في الداتابيز. مفيش عمود اسمه **Products** في جدول **Categories**. دي مجرد وسيلة راحة في الكود — EF بيتترجمها لـ **JOIN** وقت الاستعلام.

طريقتان لتعريف العلاقة

الطريقة الأولى: بالاصطلاح (تلقائي)

مجرد ما تسمي الخاصية `CategoryId` ، EF يفهم العلاقة لوحده. مش محتاج تكتب أي حاجة زيادة.

الطريقة الثانية: Fluent API (صریح)

في `AppDbContext.OnModelCreating` :

```
modelBuilder.Entity<Product>()
    .HasOne(p => p.Category)
    .WithMany(c => c.Products)
    .HasForeignKey(p => p.CategoryId)
    .OnDelete(DeleteBehavior.Cascade);
```

اقراها كجملـة إنجليزية:

الكيان `Product` له واحد (`HasOne`) من `Category` ، واللي له كتير (`WithMany`) من `Products` ، والمفتاح الأجنبي هو (`HasForeignKey`) `CategoryId` ، وعند الحذف (`OnDelete`) اعمل `Cascade` .

ليه نكتبها لو الاصطلاح كافي؟ لتلات أسباب:

1. الوضوح — أي حد يفتح الكود يشوف العلاقة صريحة
2. التحكم في `OnDelete` — الاصطلاح بيختار سلوك افتراضي، وإنـت عايز تختار بنفسك
3. الحالات المعقدة — لو عندك أكثر من علاقة بين نفس الجدولين، الاصطلاح بيتلخبط

سلوك الحذف (`OnDelete`)

السؤال: لو حذفـت تصنيف عنده ٥ منتجات، يحصل إيه للمنتجات؟

السلوك	النتيجة
Cascade	المنتجات الخمسة بتتحذف معاه (اللي إحنا مختارينه)
Restrict	الحذف بيترفض ويرمي استثناء طالما فيه منتجات
SetNull	CategoryId يبقى null (محتاج الخاصية تبقى int?)
NoAction	تسيب القرار للداتا بيز نفسها

إحنا اخترنا **Cascade** . ده مناسب هنا لأن منتج من غير تصنيف مالوش معنى. بس في أنظمة ثانية **Restrict** يبقى أمان — تخيل نظام فواتير: مش هينفع تحذف عميل وتضيع كل فواتيره.

الفهارس (Indexes)

```
modelBuilder.Entity<Category>()
    .HasIndex(c => c.Name)
    .IsUnique();
```

ال **Index** زي فهرس الكتاب — بدل ما تقلّب كل الصفحات عشان تلاقي موضوع، بتبص في الفهرس وتروح على الصفحة على طول.

IsUnique() بتضيف فائدة ثانية: بتمنع التكرار على مستوى الداتا بيز. حتى لو الكود فيه ثغرة، الداتا بيز نفسها هترفض تصنيفين بنفس الاسم.

ولاحظ إن EF عمل index ثاني لوحده:

```
CREATE INDEX "IX_Products_CategoryId" ON "Products" ("CategoryId");
```

ده تلقائي على كل Foreign Key — عشان ال **JOIN** يبقى سريع.

أنواع العلاقات الثلاثة

النوع	المثال	الملاحظة
One-to-Many ← اللي عندنا	تصنيف واحد ← منتجات كثير	ال FK في الجدول التابع
One-to-One	مستخدم واحد ← ملف شخصي واحد	ال FK بيبقى PK كمان
Many-to-Many	منتج ← وسوم كثير، ووسم ← منتجات كثير	بيحتاج جدول وسيط

في EF Core 5 وما بعده، ال Many-to-Many بقى تلقائي:

```
public class Product { public ICollection<Tag> Tags { get; set; } }
public class Tag { public ICollection<Product> Products { get; set; } }
// EF لوحده ProductTag بيعمل جدول
```

تحميل البيانات المرتبطة: ثلاث طرق

1. Eager Loading — Include (اللي بنستخدمه)

```
var products = await _context.Products.Include(p => p.Category).ToListAsync();
```

استعلام واحد فيه **JOIN**. الأفضل في معظم الحالات.

2. Explicit Loading

```
var product = await _context.Products.FindAsync(1);
await _context.Entry(product).Reference(p => p.Category).LoadAsync();
```

تحميل يدوي وقت الحاجة. مفيد لما تكون مش متأكد إنك هتحتاج البيانات المرتبطة.

3. Lazy Loading — تحميل كسول

يحتاج باكديج إضافي، وبيحمل البيانات أول ما تلمس الخاصية. **تجنبه** — بيسبب مشكلة **N+1**:

```
foreach (var p in products) // استعلام واحد يجيب ١٠٠ منتج
    Console.WriteLine(p.Category.Name); // ١٠٠! استعلام إضافي 🌟
```

جَرَب بنفسك — الفصل 7

١. روح على `/Categories/Delete/1` واحذف تصنيف "معالجات". بعدها افتح `/Products`. المنتجات اللي كانت تحتها اختفت — ده ال Cascade Delete شغال.

٢. غَيِّر `DeleteBehavior.Cascade` لـ `DeleteBehavior.Restrict`، اعمل Migration جديدة، وجَرَب تحذف تاني. إيه اللي بيحصل؟

الإجابة

هتأخذ `DbUpdateException` بسبب انتهاك قيد المفتاح الأجنبي. الداتابيز بترفض تحذف صف لسه فيه صفوف تابعة ليه. في نظام حقيقي، تمسك الاستثناء ده وتعرض رسالة مفهومة:

```
try { await _context.SaveChangesAsync(); }
catch (DbUpdateException)
{
    TempData["Error"] = "مينفعش تحذف تصنيف فيه منتجات. احذف المنتجات الأول";
    return RedirectToAction(nameof(Index));
}
```

٣. جَرَب تصنيف تصنيفين بنفس الاسم بالضبط. إيه اللي بيحصل ومين اللي منع ده؟

الإجابة

الداتابيز هترفض بسبب `IX_Categories_Name` اللي معمول `IsUnique()`. لاحظ إن الرفض جه من الداتابيز مش من كود التحقق — عشان كده الرسالة هتبقى استثناء وحش.

تمرين إضافي: ضيف تحقق في ال Controller قبل الحفظ عشان الرسالة تبقى مفهومة:

```
if (await _context.Categories.AnyAsync(c => c.Name == category.Name))
{
    ModelState.AddModelError("Name", "الاسم ده مستخدم قبل كده");
    return View(category);
}
```

٤. أضف خاصية **Description** للمنتج، ثم استخدم **Select** عشان تجيب أعمدة محددة بس:

```
var slim = await _context.Products
    .Select(p => new { p.Name, p.Price, CategoryName = p.Category.Name })
    .ToListAsync();
```

ليه ده أسرع من **Include** ؟

الإجابة

لأنه بيحجب الأعمدة اللي محتاجها بس من الداتابيز، مش كل أعمدة الجدولين. في جدول فيه ٢٠ عمود وإنت محتاج ٣، الفرق في حجم البيانات المنقولة كبير جداً.

ده الأساس اللي بيتبنى عليه مفهوم **DTO / ViewModel** — وهو خطوتك الجاية في التعلم.

الـ DbContext هو الجسر بين الكود وقاعدة البيانات. مسؤول عن ثلاث حاجات:

1. تعريف الجداول (من خلال DbSet)
2. تتبع التغييرات على الكائنات (Change Tracking)
3. ترجمة استعلامات LINQ لـ SQL

```

using Microsoft.EntityFrameworkCore;
using ShopApp.Models;

namespace ShopApp.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Product>()
            .HasOne(p => p.Category)
            .WithMany(c => c.Products)
            .HasForeignKey(p => p.CategoryId)
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<Category>()
            .HasIndex(c => c.Name)
            .IsUnique();

        modelBuilder.Entity<Category>().HasData(
            new Category { Id = 1, Name = "معالجات", Description = "CPUs", CreatedAt = new DateTime(2023, 1, 1) },
            new Category { Id = 2, Name = "كروت شاشة", Description = "GPUs", CreatedAt = new DateTime(2023, 1, 1) }
        );

        modelBuilder.Entity<Product>().HasData(
            new Product { Id = 1, Name = "Ryzen 7 7800X3D", Price = 1750m, Stock = 12, IsAvailable = true },
            new Product { Id = 2, Name = "Intel Core i5-14600K", Price = 1250m, Stock = 8, IsAvailable = true },
            new Product { Id = 3, Name = "RTX 4070 Super", Price = 2600m, Stock = 5, IsAvailable = true }
        );
    }
}

```

سطر بسطر

public class AppDbContext : DbContext

 بنرت من **DbContext** — الكلاس الأساسي في EF Core اللي فيه كل منطق التنجع والترجمة.

```
public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
{
}
```

ال Constructor ده فاضي عن قصد. كل شغله إنه يستقبل الإعدادات (options) ويمزرها للكلاس الأب بـ : base(options) .

فين الإعدادات دي بتتحدد؟ في Program.cs :

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(connectionString));
```

ليه الطريقة دي أحسن من كتابة الإعدادات جوه ال Context؟

الطريقة القديمة كانت:

```
protected override void OnConfiguring(DbContextOptionsBuilder options)
{
    options.UseSqlite("Data Source=shopapp.db"); // مبروط بالكود ✗
}
```

المشكلة: ال connection string اتحول لجزء من الكود. مش هتقدر تغيره بين بيئة التطوير والإنتاج، ومش هتقدر تستبدله بداتايبز وهمية (In-Memory) وقت الاختبار.

بالطريقة اللي إحنا مستخدمينها، تقدر تعمل اختبار كده:

```
var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseInMemoryDatabase("TestDb") // داتايبز في الذاكرة للاختبار
    .Options;

using var context = new AppDbContext(options);
// اختبر براحتك من غير ما تلمس الداتايبز الحقيقية
```

```
public DbSet<Category> Categories => Set<Category>();
```

كل <T> DbSet = جدول في الداتايبز. اسم الخاصية بيبقى اسم الجدول:

الخاصية	الجدول
Categories	Categories
Products	Products

إيه الفرق بين الطريقتين دول؟

```
public DbSet<Category> Categories { get; set; } // الطريقة التقليدية
public DbSet<Category> Categories => Set<Category>(); // اللي إحنا مستخدمينه
```

الثانية بتستخدم **Expression-bodied property** (`=>`) وبتنادي `Set<T>()` من الكلاس الأب. الفائدة: مفيش تحذير من ال-Nullable، لأن مفيش خاصية ممكن تفضل `null` — القيمة بتتسبب في كل مرة.

OnModelCreating

الميثود دي بتننقذ مرة واحدة لما EF يبني نموذج الداتابيز في ذاكرته. هنا بتتطلب أي حاجة مش مغطاة بالاصطلاح.

في الأول — عشان تسبب EF يطبق قواعده الافتراضية قبل ما تضيف قواعدهك. `base.OnModelCreating(modelBuilder)`

HasData — البيانات المبدئية

```
modelBuilder.Entity<Category>().HasData(
    new Category { Id = 1, Name = "معالجات", ... }
);
```

دي **Seed Data** — بيانات بتنزرع مع ال-Migration. لاحظ ثلاث شروط مهمة:

١. لازم تحدد **Id** صراحة. عادةً **Id** بيتولد تلقائياً، لكن في ال-Seed لازم تحده لأن EF بيستخدمه عشان يقارن بين Migration والثانية ويعرف إيه اللي اتغير.

٢. لازم القيم تكون ثابتة **(Deterministic)**. ده غلط:

```
CreatedAt = DateTime.Now // بتتغير في كل مرة ❌
```

لأن EF بيقرن القيم بين ال-Migrations. لو القيمة بتتغير، EF هيفتكر إن فيه تعديل ويعمل Migration جديدة كل مرة. عشان كده كتبناها ثابتة:

```
CreatedAt = new DateTime(2025, 1, 1) // ثابتة ✓
```


٣. البيانات بتتزرع مع ال Migration مش وقت التشغيل. شوف SQL اللي اتولّد في ملف ال Migration:

```
INSERT INTO "Categories" ("Id", "CreatedAt", "Description", "Name")
VALUES (1, '2025-01-01 00:00:00', 'CPUs', 'معالجات');
```

تتبع التغييرات (Change Tracking)

ده أذكى جزء في EF Core. جَرَب تفهم الكود ده:

```
var product = await _context.Products.FirstAsync(p => p.Id == 1);
product.Price = 1900; // مجرد تعديل على كائن عادي في الذاكرة
await _context.SaveChangesAsync(); // EF انغير! عرف لوحده إن السعر انغير!
```

إزاي عرف؟ لما جبت المنتج، EF خزن لقطة (Snapshot) من قيمه الأصلية. وقت `SaveChanges` بيقرن الحالي بالأصلي، ويولد `UPDATE` للأعمدة المتغيرة بس:

```
UPDATE "Products" SET "Price" = 1900 WHERE "Id" = 1;
-- مش في الأمر لأنهم ما اتغيروش Stock و Name: لاحظ --
```

AsNoTracking() — متى ولماذا

```
await _context.Products.AsNoTracking().ToListAsync();
```

بتقول لـ EF: "مش هعدّل على البيانات دي، متتعيش نفسك بالتتبع".

الفائدة: أسرع وبتاكل ذاكرة أقل — ممكن الفرق يوصل ٣٠% في القوائم الكبيرة.

القاعدة:

الحالة	استخدم
عرض بيانات فقط (Index, Details)	✓ <code>AsNoTracking()</code>
هتعدّل وتحفظ (Edit)	تتبع عادي (بدون <code>AsNoTracking</code>)

شوف في مشروعنا الفرق:

```
// GetAllAsync - للعرض بس
public async Task<IEnumerable<Product>> GetAllAsync() =>
    await _context.Products.Include(p => p.Category)
        .AsNoTracking() // عرض فقط ←
        .ToListAsync();

// GetByIdAsync - ممكن نعدّل عليه بعدها
public async Task<Product?> GetByIdAsync(int id) =>
    await _context.Products.Include(p => p.Category)
        .FirstOrDefaultAsync(p => p.Id == id); // بدون AsNoTracking
```

جرب بنفسك — الفصل 8

١. ضيف تصنيف جديد في **HasData** :

```
new Category { Id = 3, Name = "رامات", Description = "RAM", CreatedAt = new DateTime(2025, 1, 1)
```

اعمل Migration واقرا الملف المتولد — هتلاقي **InsertData** واحدة بس للتصنيف الجديد، مش الكل. EF قارن وعرف الفرق لوحده.

٢. غير **CreatedAt = new DateTime(2025, 1, 1)** لـ **DateTime.Now** واعمل Migration مرتين ورا بعض. هتلاحظ إيه؟

الإجابة

كل مرة هتلاقي Migration جديدة فيها **UpdateData** للتاريخ — حتى لو ما غيرتش أي حاجة. لأن **DateTime.Now** بترجع قيمة مختلفة في كل تنفيذ، فEF بيفتكر إن فيه تعديل.

الدرس: بيانات الـ **Seed** لازم تكون ثابتة دائماً.

٣. أضف السطر ده مؤقتاً في **ProductService.GetAllAsync** قبل **ToToListAsync** :

```
.Where(p => p.Price > 1500)
```

شغل وشوف الـ SQL في الترمينال. هتلاقي **WHERE "p"."Price" > 1500** — يعني الفترة انتقذت في الداتايز مش في ذاكرة التطبيق. دي قوة LINQ to Entities.

٤. دلوقتي جرب الفرق ده:

```
// أ - الفلتر في الداتايز
var a = await _context.Products.Where(p => p.Price > 1500).ToListAsync();

// ب - الفلتر في الذاكرة
var b = (await _context.Products.ToListAsync()).Where(p => p.Price > 1500);
```

اللاتين بيرجعوا نفس النتيجة. ليه (أ) أفضل بكثير؟

الإجابة

في (أ): الداتايز بترجع المنتجات المطابقة بس.

في (ب): الداتايز بترجع كل المنتجات للتطبيق، وبعدين التطبيق بيفلتر. لو عندك مليون منتج، هتنقل مليون صف على الشبكة عشان تستخدم ١٠٠ منهم.

السبب التقني: `ToListAsync()` بينفذ الاستعلام فوراً. أي `Where` بعد كده بيشتغل على `List` في الذاكرة (LINQ to Objects) مش على الداتايز (LINQ to Entities).

القاعدة: حط كل الفلتر والترتيب قبل `ToListAsync()`.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=shopapp.db",
    "_SqlServerExample": "Server=(localdb)\\MSSQLLocalDB;Database=ShopAppDb;Trusted_Connection=Tr
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

ConnectionStrings

بتتقرا في `Program.cs` كده:

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
```

الميثود `GetConnectionString("X")` هي اختصار لـ `Configuration["ConnectionStrings:X"]`.

Logging.LogLevel

بتتحكم في كمية الرسائل اللي بتظهر في الترمينال. المستويات من الأقل للأعلى:

```
Trace < Debug < Information < Warning < Error < Critical
```

لما تحدد مستوى، بتشوف المستوى ده وكل اللي فوقه.

السطر ده تحديداً هو سبب سيل رسائل SQL اللي شفتها:

```
"Microsoft.EntityFrameworkCore.Database.Command": "Information"
```

حظيته عن قصد عشان تشوف الـ SQL اللي EF بيولده — ده مفيد جداً وإنك بتتعلم. لما تزهق، غيره لـ **"Warning"**.

ترتيب أولوية الإعدادات

ASP.NET بيقرأ الإعدادات من مصادر متعددة بالترتيب ده، والمتأخر بيغطي على المتقدم:

الأولوية	المصدر	ملاحظة
١ (الأدنى)	<code>appsettings.json</code>	الأساس
٢	<code>appsettings.{Environment}.json</code>	حسب البيئة — Development / Production
٣	User Secrets	أسرار محلية، بيئة التطوير فقط
٤	متغيرات البيئة (Environment Variables)	الشائع في السيرفرات
٥ (الأعلى)	معطيات سطر الأوامر	بتغطي على الكل

مثال عملي: لو `appsettings.json` فيه `"Default": "Information"` و `appsettings.Development.json` فيه `"Default": "Debug"`، فوقت التطوير هستخدم `Debug`.

⚠ تحذير أمني مهم

متحطش أي كلمات سر أو مفاتيح API في `appsettings.json` — الملف ده بيترفع على Git وأي حد يشوف الريبو هيشوفهم. للتطوير المحلي استخدم User Secrets:

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=...;Password=SuperSecret123"
```

بيتخزن بره فولدر المشروع تماماً، فمستحيل يترفع على Git بالغلط.

للإنتاج استخدم متغيرات البيئة أو خدمة إدارة أسرار (Azure Key Vault, AWS Secrets Manager).

جَرِّبْ بِنَفْسِكَ — الفصل 9

١. غَيِّر `"Microsoft.EntityFrameworkCore.Database.Command"` لـ `"Warning"` وشغِّل الترمينال هيبقى نضيف. رجِّعه لما تحتاج تشوف SQL تاني.

٢. أضف قسم إعدادات مخصص:

```
"AppSettings": {  
  "SiteName": "متجر PC ELITES",  
  "ItemsPerPage": 10  
}
```

واقراه في أي كمنترولر:

```
public class ProductsController : Controller  
{  
    private readonly IConfiguration _config;  
    public ProductsController(IConfiguration config) => _config = config;  
  
    public IActionResult Index()  
    {  
        ViewBag.SiteName = _config["AppSettings:SiteName"];  
        var perPage = _config.GetValue<int>("AppSettings:ItemsPerPage");  
        // ...  
    }  
}
```

لاحظ إن `IConfiguration` وصلت للكمترولر بنفس طريقة الـ DI.

٣. افتح `appsettings.Development.json` وضيف فيه:

```
"AppSettings": { "SiteName": "متجر - بيئة تطوير" }
```

شغِّل. أنهى قيمة ظهرت وليه؟

الفصل 10: Dependency Injection — الفصل الأهم

المشكلة أولاً

تخيل كمنترولر يعمل احتياجاته بنفسه:

```
public class ProductsController : Controller
{
    private readonly ProductService _service;

    public ProductsController()
    {
        var context = new AppDbContext(/* إعدادات من فين؟ */);
        _service = new ProductService(context); // ❌
    }
}
```

أربع مشاكل قاتلة:

المشكلة	التفسير
ارتباط وثيق	المنترولر مربوط بـ ProductService بالاسم. تغييره = تعديل المنترولر
استحالة الاختبار	مينفكش تستبدل الـ Service بنسخة وهمية في الاختبار
تكرار الإعدادات	كل منترولر لازم يعرف الـ connection string
مفيش إدارة للعمر	مين هيقل الـ DbContext ؟ ومتى؟

الحل: اقلب المسؤولية

بدل ما المنترولر يعمل احتياجاته، حد ثاني يعملها له ويديها له.

ده اسمه **Inversion of Control (IoC)**، وتطبيقه العملي اسمه **Dependency Injection**.

```
public class ProductsController : Controller
{
    private readonly IProductService _productService;

    // يجهز ASP.NET و - "IProductService" بس أقول "أنا محتاج"
    public ProductsController(IProductService productService)
    {
        _productService = productService;
    }
}
```

لاحظ حاجتين:

1. الكنترولر يطلب `IProductService` (الواجهة) مش `ProductService` (الكلاس). فهو ما يعرفش أصلاً مين هينفذ.
2. مفيش `new` خالص. الكنترولر بس بيستقبل.

التسجيل في `Program.cs`

```
builder.Services.AddScoped<IProductService, ProductService>();
```

اقرأها: "لما حد يطلب `IProductService` ، اديله نسخة من `ProductService`".

الأعمار الثلاثة — بالتفصيل

`AddSingleton` — نسخة واحدة للأبد

```
builder.Services.AddSingleton<ICacheService, MemoryCacheService>();
```

App Start $\Longrightarrow$ [ONE INSTANCE] $\Longrightarrow$ App Stop

نسخة واحدة بتتعمل أول مرة وبتفضل لآخر التطبيق. كل المستخدمين بيشاركوا نفس النسخة.

متى؟ الكاش، قراءة الإعدادات، أي حاجة ثابتة وثقيلة الإنشاء.

⚠️ خطر: لازم تكون **Thread-Safe** لأن آلاف الطلبات ممكن يستخدموها في نفس اللحظة. ومتحطش فيها بيانات خاصة بمستخدم معين.


```
builder.Services.AddScoped<IProductService, ProductService>();
```

```
Request #1  |— [ instance A ] —|
Request #2  |— [ instance B ] —|
Request #3  |— [ instance C ] —|
```

كل طلب لديه نسخته الخاصة، ويتموت بموت الطلب.

نسخة جديدة مع كل طلب HTTP، وبتتخلص لما الطلب يخلص.

متى؟ **DbContext** والخدمات التي تستخدمه. ده الاختيار الافتراضي في ٩٠٪ من الحالات.

نقطة مهمة: لو طلبت نفس الخدمة مرتين في نفس الطلب، هتأخذ نفس النسخة. عشان كده في **ProductsController**:

```
public ProductsController(IProductService productService, ApplicationDbContext context)
```

ال **context** اللي في الكنترولر هو نفس ال **context** اللي جوه **ProductService**. نفس النسخة، نفس التتبع، نفس المعاملة (Transaction).

```
builder.Services.AddTransient<IEmailSender, SmtpEmailSender>();
```

Inside ONE request:

```
1st resolve → [ instance A ]
2nd resolve → [ instance B ]
3rd resolve → [ instance C ]
```

جوه الطلب الواحد نفسه، كل مرة بتطلبها بتأخذ نسخة جديدة.

متى؟ خدمات خفيفة ومالهاش حالة (Stateless).

جدول القرار السريع

الحالة	العمر
DbContext أو أي حاجة بتستخدمه	Scoped
كاش أو إعدادات ثابتة	Singleton
خدمة خفيفة بلا حالة	Transient
مش عارف؟	Scoped

⚠ الخطأ القاتل: Captive Dependency

متسجلش خدمة عمرها قصير جوة خدمة عمرها طويل.

```
builder.Services.AddSingleton<IProductService, ProductService>(); // كارثة ✗
// ProductService بيأخد ApplicationDbContext (Scoped)
```

اللي هيحصل: الـ **DbContext** هيتأسر جوة الـ **Singleton** ويفضل عايش للأبد. النتيجة:

- تسريب ذاكرة (كل الكيانات المتتبعة تفضل في الذاكرة)
- بيانات قديمة (الـ Context مش هيشوف تعديلات الطلبات الثانية)
- انهيارات عشوائية (**DbContext** مش Thread-Safe)

القاعدة: الخدمة ممكن تعتمد على خدمة عمرها مساوي أو أطول، مش أقصر.

الخدمة	ممكن تعتمد على
Singleton	Singleton فقط
Scoped	Singleton و Scoped
Transient	الكل

الخبر الحلو: ASP.NET Core بيكتشف الخطأ ده وقت التشغيل في بيئة التطوير ويرمي استثناء واضح.

جرب بنفسك — الفصل 10

١. جرب الخطأ عن قصد — غير التسجيل لـ:

```
builder.Services.AddSingleton<IProductService, ProductService>();
```

شغل واقرأ رسالة الخطأ. هتلاقيها بتقول:

```
Cannot consume scoped service 'AppDbContext' from singleton 'IProductService'
```

رسالة واضحة جداً. رجّعها لـ **AddScoped**.

٢. أثبت إن **Scoped** فعلاً نسخة واحدة لكل طلب. ضيف في **ProductService**:

```
public ProductService(AppDbContext context)
{
    _context = context;
    Console.WriteLine($"ProductService instance: {GetHashCode()}");
}
```

وفي الكنترولر:

```
public ProductsController(IProductService productService, AppDbContext context)
{
    _productService = productService;
    _context = context;
    Console.WriteLine($"Controller got context: {context.GetHashCode()}");
}
```

افتح **/Products** مرتين. هتلاحظ إن الأرقام بتتغير بين الطلبين، لكن جوّه الطلب الواحد الأرقام ثابتة.

٣. غيرّها لـ **AddTransient** وكّر التجربة. إيه اللي اتغير؟

٤. أضف خدمة جديدة بنفسك:

```
// Services/ICategoryService.cs
public interface ICategoryService
{
    Task<IEnumerable<Category>> GetAllWithProductCountAsync();
}
```

نفّذها، سجّلها في **Program.cs**، واستخدمها في **CategoriesController** بدل الـ **DbContext** المباشر.

ليه طبقة زيادة أصلاً؟

ال `CategoriesController` يستخدم `AppDbContext` مباشرة، وال `ProductsController` يستخدم `IService`. الاتنين شغالين. فليه التعقيد الزيادة؟

ثلاث أسباب:

1. عدم تكرار الكود

تخيل إنك محتاج "المنتجات المتاحة والمخزون فيها أكثر من صفر" في: - صفحة المنتجات - ال API - تقرير شهري - صفحة البحث من غير `Service`، هتكرر نفس الاستعلام ٤ مرات. مع `Service`، بتكتبه مرة:

```
public async Task<IEnumerable<Product>> GetAvailableAsync() =>
    await _context.Products
        .Where(p => p.IsAvailable && p.Stock > 0)
        .Include(p => p.Category)
        .AsNoTracking()
        .ToListAsync();
```

2. قابلية الاختبار

الكنترولر بيعتمد على `IService` (واجهة). في الاختبار تقدر تستبدلها بنسخة وهمية:

```
var fake = new Mock<IService>();
fake.Setup(s => s.GetAllAsync()).ReturnsAsync(new List<Product>
{
    new Product { Id = 1, Name = "منتج تجريبي", Price = 100 }
});

var controller = new ProductsController(fake.Object, null!);
var result = await controller.Index();

// اختبرت الكنترولر من غير داتايزر خالص
```

لو الكنترولر كان بياخد `AppDbContext` مباشرة، ده كان مستحيل.

3. تركيز منطق العمل في مكان واحد

قواعد زي "مينفعش تحذف منتج لو عليه طلبات معلقة" مكانها ال Service، مش الكنترولر. الكنترولر شغله استقبال الطلب والرد بس.

العقد — IProductService.cs

```
public interface IProductService
{
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(int id);
    Task AddAsync(Product product);
    Task UpdateAsync(Product product);
    Task DeleteAsync(int id);
    Task<bool> ExistsAsync(int id);
}
```

الواجهة بتقول "إيه" من غير "إزاي". أي كلاس ينقّذها بيبقى صالح للاستخدام.

ليه `Task<...>` في كل حاجة؟ لأن كل العمليات غير متزامنة (`async`).

لحظة — إيه هو `async` أصلاً؟

```
// متزامن - الخيط بيوقف مستني
var products = _context.Products.ToList();

// غير متزامن - الخيط بيسيب مكانه لطلبات ثانية
var products = await _context.Products.ToListAsync();
```

الفرق العملي:

استعلام الداتا بيز بياخد مثلاً ٥٠ ملي ثانية. في الطريقة المتزامنة، الخيط (Thread) بيفضل واقف مستني ال ٥٠ ملي ثانية دول من غير ما يعمل حاجة.

في الطريقة غير المتزامنة، الخيط بيرجع لمجمّع الخيوط (Thread Pool) ويخدم طلبات ثانية، ولما الداتا بيز ترد بيرجع يكمل.

النتيجة: سيرفر بـ ١٠٠ خيط يقدر يخدم آلاف الطلبات المتزامنة بدل ١٠٠ بس.

القاعدة الذهبية: `async` طول السلسلة. لو ال Service بترجع `Task`، الكنترولر لازم يكون `async` كمان:

```
public async Task<IActionResult> Index()
{
    var products = await _productService.GetAllAsync();
    return View(products);
}
```

⚠ متعاملش **.Result** أو **.Wait()** أبداً:

```
var products = _productService.GetAllAsync().Result; // ❌ ممكن يعلق التطبيق (Deadlock)
```

التنفيذ — ProductService.cs

```
public class ProductService : IProductService
{
    private readonly ApplicationDbContext _context;

    public ProductService(ApplicationDbContext context)
    {
        _context = context; // ← DI تاني! الـ DI
    }
}
```

لاحظ إن الـ DI مش بس للكونترولرات — أي كلاس مسجل في الـ Container بيقدري استقبال احتياجاته بنفس الطريقة.

```
public async Task<IEnumerable<Product>> GetAllAsync() =>
    await _context.Products
        .Include(p => p.Category) // مع التصنيفات JOIN
        .OrderBy(p => p.Name) // ترتيب في الداتايز
        .AsNoTracking() // عرض فقط، بلا تتبع
        .ToListAsync(); // تنفيذ الاستعلام
```

ترتيب السطور مقصود: كل السطور اللي قبل **ToListAsync()** بتبني الاستعلام بس من غير ما تنفذه. **ToListAsync()** هي اللي بتبع SQL فعلاً.

ده اسمه **Deferred Execution** — التنفيذ المؤجل.

```
public async Task DeleteAsync(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product is null) return; // مش موجود؟ اخرج بهدوء

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();
}
```

ليه **FindAsync** مش **FirstOrDefaultAsync** ؟

FindAsync بتدور في الذاكرة المحلية الأول. لو الكيان اتحمل قبل كده في نفس الطلب، بترجعه من غير ما تروح الداتابيز خالص. أسرع لما تكون بتدور بال Primary Key.

جرب بنفسك — الفصل 11

١. ضيف ميثود جديدة للواجهة:

```
Task<IEnumerable<Product>> SearchAsync(string term);
```

نفذها:

```
public async Task<IEnumerable<Product>> SearchAsync(string term) =>
    await _context.Products
        .Include(p => p.Category)
        .Where(p => p.Name.Contains(term))
        .AsNoTracking()
        .ToListAsync();
```

واستخدمها في action جديد اسمه **Search**. لاحظ إن الكومبايلر أجبرك تنفذ الميثود في **ProductService** أول ما ضفتها للواجهة — دي قوة العقود.

٢. غير **GetAllAsync** تستخدم **FirstOrDefault** بدل **Include** وشوف الـ SQL في الترمينال. قارن عدد الاستعلامات.

٣. حول **CategoriesController** كله يستخدم **ICategoryService** بدل **AppDbContext**. ده تمرين متكامل هيثبت كل اللي فات.

الفصل 12: Program.cs — نقطة التجميع

الملف ده هو قلب التطبيق. مقسوم لأربع مراحل بترتيب صارم.

المرحلة صفر: البداية

```
var builder = WebApplication.CreateBuilder(args);
```

السطر ده بيعمل حاجات كتير خلف الكواليس:

- يقرأ `appsettings.json` و `appsettings.Environment.json`
- يقرأ متغيرات البيئة ومعطيات سطر الأوامر
- يجهّز نظام التسجيل (Logging)
- يجهّز سيرفر Kestrel
- يعمل الـ **DI Container** الفاضي

المرحلة 1: تسجيل الخدمات

```
builder.Services.AddControllersWithViews();
```

بتسجّل كل ما يلزم لـ MVC: اكتشاف الكنترولرات، محرّك Razor، Model Binding، التحقق، Tag Helpers.

فيه أخوات ليها: `AddControllers()` لـ Web API بلا Views، و `AddRazorPages()` لنمط Razor Pages.

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
?? throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");
```

المعامل `??` اسمه **Null-Coalescing Operator**. معناه: "لو اللي على الشمال `null`، نفّذ اللي على اليمين".

فبدل ما التطبيق يقع بعددين برسالة غامضة، بيوقع دلوقتي برسالة واضحة. المبدأ ده اسمه **Fail Fast** — اكتشاف الخطأ بأسرع ما يمكن.

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(connectionString));
```

الميثود `AddDbContext` بتعمل ثلاث حاجات: 1. تسجل `AppDbContext` كـ **Scoped** تلقائياً 2. تسجل `DbContextOptions<AppDbContext>` بالإعدادات المحددة 3. تظبط تجميع الاتصالات (Connection Pooling)

```
builder.Services.AddScoped<IProductService, ProductService>();
```

خدمتنا المخصصة.

```
var app = builder.Build();
```

نقطة الارجعة. بعد السطر ده، ال Container اتقفل ومينفعش تسجل خدمات جديدة.

المرحلة 2: ال Middleware Pipeline

```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
else
{
    app.UseDeveloperExceptionPage();
}
```

ليه الاختلاف بين البيئتين؟

في التطوير: `UseDeveloperExceptionPage()` بتعرض تفاصيل الخطأ كاملة — ال Stack Trace، السطر اللي وقع، القيم. ممتاز للتشخيص.

في الإنتاج: نفس الصفحة دي **ثغرة أمنية خطيرة** — بتكشف بنية مشروعك ومسارات ملفاتك وأحياناً بيانات حساسة. عشان كده بنعرض صفحة خطأ عامة بدلها.

`UseHsts()` بتبعت هيدر بيقول للمتصفح: "المرات الجاية استخدم HTTPS من الأول ومتحاولش HTTP خالص".

```
app.UseHttpsRedirection(); // http → https
app.UseStaticFiles(); // ملفات wwwroot
app.UseRouting(); // تحليل ال URL
app.UseAuthorization(); // فحص الصلاحيات
```

الترتيب ده مش عشوائي:

- `UseStaticFiles` قبل `UseRouting` عشان طلبات الـ CSS والصور تخرج بدري من غير ما تدخل نظام الراوتنج (أداء أفضل)
- `UseAuthorization` لازم بعد `UseRouting` عشان يعرف المستخدم رايح فين

المرحلة 3: الراوتنج

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Products}/{action=Index}/{id?}");
```

تفكيك القالب:

الجزء	المعنى
<code>{controller=Products}</code>	متغير اسمه <code>controller</code> , افتراضيه <code>Products</code>
<code>{action=Index}</code>	متغير اسمه <code>action</code> , افتراضيه <code>Index</code>
<code>{id?}</code>	متغير اسمه <code>id</code> , اختياري (علامة <code>?</code>)

أمثلة تطبيقية:

URL	controller	action	id
/	Products	Index	—
/Categories	Categories	Index	—
/Products/Create	Products	Create	—
/Products/Edit/5	Products	Edit	5

المرحلة 4: ال Migration التلقائي

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}
```

ليه محتاجين `CreateScope()` ؟

`AppDbContext` مسجل كـ **Scoped** — يعني بيعيش داخل نطاق طلب HTTP. لكن إحنا هنا قبل ما أي طلب يوصل. فلازم نعمل نطاق يدوي.

كلمة `using` بتضمن إن النطاق يتقفل وال Context يتخلص منه بعد ما نخلص.

`db.Database.Migrate()` بتطبق أي Migrations لسه ما اتطبقتش. دي اللي شفت أثرها في الترمينال أول ما شغلت المشروع.

⚠ ملاحظة للإنتاج: الطريقة دي مريحة للتعليم، لكن في السيرفرات الحقيقية بيُفَضَّل تطبيق ال Migrations كخطوة منفصلة في خط النشر (CI/CD)، مش وقت تشغيل التطبيق. السبب: لو عندك أكثر من نسخة من التطبيق شغالة، هيحاولوا يطبقوا ال Migration في نفس الوقت.

```
app.Run();
```

بيشغل السيرفر ويستنى الطلبات. الكود ده بيوقف هنا لحد ما توقف التطبيق بـ `Ctrl+C`.

جرب بنفسك — الفصل 12

١. غيّر الراوت الافتراضي لـ `Categories`:

```
pattern: "{controller=Categories}/{action=Index}/{id?}"
```

شغل وافتح `localhost:5124`. هتروح فين؟

٢. أضف راوت مخصص قبل الافتراضي:

```
app.MapControllerRoute(
    name: "productDetails",
    pattern: "منتج/{id}",
    defaults: new { controller = "Products", action = "Details" });
```

جَرِّب `localhost:5124/1/منتج` . الراوتنج يبدعم العربي عادي.

٣. شيل `app.UseStaticFiles();` وشغّل. الصفحة هتظهر من غير تنسيق خالص — لأن `bootstrap.rtl.min.css` بقى مش قابل للوصول.

٤. جَرِّب تسجّل خدمة بعد `builder.Build()` :

```
var app = builder.Build();
builder.Services.AddScoped<IProductService, ProductService>(); // ❌
```

شغّل واقرا رسالة الخطأ. ليه ممنوع؟

الإجابة

```
InvalidOperationException: Cannot modify ServiceCollection after
the application has been built.
```

لأن `Build()` بتحوّل قائمة الخدمات لـ `Container` محسّن وجاهز. لو سمح بالتعديل بعدها، الخدمات اللي اتعملت بالفعل مش هتشوف التغيير — وده هيسبب سلوك غير متوقع صعب تشخيصه. المنع صراحةً أأمن.

٥. جَرِّب تشيل بلوك الـ `Migration` التلقائي، امسح `shopapp.db` ، وشغّل. إيه اللي هيحصل؟ وإزاي تحل من غير ما ترجع الكود؟

الإجابة

هتأخذ خطأ `no such table: Products` لأن الداتابيز فاضية.

الحل من غير الكود: طبّق الـ `Migration` يدوي:

```
dotnet ef database update
```

ده بيوضّح إن البلوك ده مجرد وسيلة راحة — الـ `Migration` ممكن يتطبق يدوي برضه.

هيكل الكنترولر

```
public class CategoriesController : Controller
{
    private readonly ApplicationDbContext _context;

    public CategoriesController(ApplicationDbContext context)
    {
        _context = context;
    }
    // ... Actions
}
```

Controller : — بنرت من الكلاس الأساسي اللي بيدينا ميثودات جاهزة زي `View()` و `RedirectToAction()` و `NotFound()`، وخصائص زي `ModelState` و `TempData` و `HttpContext`.
private readonly — كلمة `readonly` بتمنع تغيير المتغير بعد الـ Constructor. ضمان إضافي إن الـ Context مش هيتبدل بالغلط في نص الكود.

نمط الـ CRUD السباعي

كل كنترولر الـ CRUD في MVC بيتكون من سبع **actions** بالنمط ده:

#	Action	HTTP	الوظيفة
1	Index	GET	عرض القائمة
2	Details/5	GET	عرض عنصر واحد
3	Create	GET	عرض فورم فاضي
4	Create	POST	استقبال البيانات وحفظها
5	Edit/5	GET	عرض فورم متعبي
6	Edit/5	POST	استقبال التعديلات وحفظها
7	Delete/5	GET + POST	تأكيد ثم حذف

ليه كل عملية لها GET و POST؟

- **GET** = "وريني الفورم" — عملية آمنة، مش بتغير حاجة
- **POST** = "خد البيانات دي واحفظها" — عملية بتغير الحالة

الفصل ده مهم لأن المتصفحات والبروكسيات بتفترض إن GET آمن. لو خليت الحذف على GET، أي زاحف (Crawler) أو أداة تسريع بتفتح الروابط مسبقاً ممكن تمسح بياناتك كلها!

1. Index — القائمة

```
public async Task<IActionResult> Index()
{
    var categories = await _context.Categories
        .Include(c => c.Products)
        .AsNoTracking()
        .ToListAsync();

    return View(categories);
}
```

Task<IActionResult> — نوع الإرجاع. **IActionResult** واجهة عامة بتسمح بإرجاع أي نتيجة:

```

return View(model);           // IActionResult → HTML
return RedirectToAction("X"); // IActionResult → 302
return NotFound();           // IActionResult → 404
return Json(data);           // IActionResult → JSON
return File(bytes, "app/pdf"); // IActionResult → تحميل ملف
return Content("نص");         // IActionResult → نص خام

```

هترجع `item.Products.Count` — عشان نقدر نعرض عدد المنتجات في كل تصنيف. من غيرها `Include(c => c.Products)` صفر دائماً.

`return View(categories)` — بيدور على `Views/Categories/Index.cshtml` (بالاصطلاح: فولدر باسم الكنترولر، ملف باسم الـ Action) ويمزق القائمة كموديل.

2. Details — عنصر واحد

```

public async Task

```

ليه `int?` بعلامة استفهام؟

لأن الـ `id` في الراوت اختياري (`{id?}`). لو المستخدم كتب `/Categories/Details` من غير رقم، `id` هتبقى `null` بدل ما يقع التطبيق.

فحصين منفصلين مقصودين:

الحالة	الفحص
المستخدم مبعثش رقم أصلاً	<code>id is null</code>
بعت رقم لكن مش موجود في الداتابيز	<code>category is null</code>

`FirstOrDefaultAsync` بترجع أول نتيجة أو `null`. أخواتها:

الميثود	السلوك لو مفيش نتيجة	لو أكثر من نتيجة
FirstOrDefaultAsync	null	يرجع الأول
FirstAsync	استثناء	يرجع الأول
SingleOrDefaultAsync	null	استثناء
SingleAsync	استثناء	استثناء

استخدم **Single*** لما تكون متأكد إن فيه نتيجة واحدة بالضبط — بتكشف الأخطاء في البيانات.

3. Create (GET) — الفورم الفاضي

```
public IActionResult Create() => View();
```

أبسط action في المشروع. مفيش **async** لأن مفيش عمليات داتا بيز — بس بيعرض فورم فاضي.

=> View() — Expression-bodied method, اختصار لـ **{ return View(); }**.

4. Create (POST) — الحفظ

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("Name,Description")] Category category)
{
    if (!ModelState.IsValid) return View(category);

    _context.Add(category);
    await _context.SaveChangesAsync();
    TempData["Success"] = "تم إضافة التصنيف بنجاح";
    return RedirectToAction(nameof(Index));
}
```

[HttpPost]

بتحدد إن الـ action دي بتد على POST بس. عشان كده ينفع يكون عندنا **Create()** مرتين بنفس الاسم — واحدة GET واحدة POST. الفرق في الـ HTTP Verb.

إيه هو هجوم CSRF؟

1. إنت مسجل دخول في موقعك، والكوكيز محفوظة في المتصفح
2. تفتح موقع خبيث في تاب ثاني
3. الموقع الخبيث فيه فورم مخفي بيبعت POST لموقعك
4. المتصفح بيبعت الكوكيز بتاعتك تلقائياً مع الطلب
5. موقعك بينفذ العملية وهو فاكر إنك إنت اللي طلبتها 🌟

إزاي التوكن بيمنع ده؟

- وقت عرض الفورم، السيرفر بيولد توكن عشوائي ويحطه في حقل مخفي وفي كوكي
- وقت الإرسال، لازم الاتنين يتطابقوا
- الموقع الخبيث مش قادر يقرأ الحقل المخفي بسبب سياسة Same-Origin

فين الحقل المخفي؟ بيتولد تلقائياً مع أي `<form>` فيها `asp-action`. افتح الصفحة في المتصفح واعمل View Source، هتلاقي:

```
<input name="__RequestVerificationToken" type="hidden" value="CfDJ8Nr..." />
```

Over-posting من حماية — [Bind("Name,Description")]

إيه هي مشكلة الـ Over-posting؟

تخيل موديل فيه:

```
public class User
{
    public string Name { get; set; }
    public bool IsAdmin { get; set; } // خطير ←
}
```

والفورم فيه حقل الاسم بس. مستخدم ذكي يقدر يفتح أدوات المطور ويضيف حقل يدوي:

```
<input name="IsAdmin" value="true" />
```

الـ Model Binder هيملا `IsAdmin = true` وهو مبسوط، والمستخدم بقى أدمن! 🌟

[Bind] بتقول: "املا الخصائص دي بس، وتجاهل أي حاجة ثانية جاية من الفورم".

شوف الفرق بين الـ Create والـ Edit في مشروعنا:

```
// Create – مفيش Id تلقائياً لأنه بيتولد تلقائياً
[Bind("Name,Description")]

// Edit – Id من نعدّل معرف محتاجينه نعرف نعدّل من Id
[Bind("Id,Name,Description,CreatedAt")]
```

ملاحظة: [Bind] حل سريع ومناسب للمشاريع الصغيرة. الحل الاحترافي هو **ViewModels** — كلاسات منفصلة فيها الحقول المسموح بيها بس. هنتكلم عنها في الفصل ١٩.

```
if (!ModelState.IsValid) return View(category);
```

ModelState بيتملي تلقائياً بنتائج التحقق من الـ Data Annotations. لو حصل خطأ:

```
return View(category); // نرجع نفس الفورم بالبيانات اللي المستخدم كتبها
```

نقطة مهمة: بنمرّر **category** ثاني عشان المستخدم ميعيدش كتابة كل حاجة. الأخطاء بتظهر تلقائياً في `<span asp-validation-for="...">`.

```
TempData["Success"]
```

الفرق بين طرق تمرير البيانات لـ **View**:

الطريقة	العمر	الاستخدام
ViewBag / ViewData	الطلب الحالي فقط	تمرير بيانات لـ View
TempData	يعيش عبر Redirect واحد	رسائل بعد إعادة توجيه
Session	كل الجلسة	بيانات المستخدم

بنستخدم **TempData** هنا بالظبط لأن بعد الحفظ فيه **Redirect**. لو استخدمنا **ViewBag** الرسالة كانت هتضيع.

والرسالة بتتعرض في `_Layout.cshtml`:

```
@if (TempData["Success"] is string msg)
{
    <div class="alert alert-success">@msg</div>
}
```

```
return RedirectToAction(nameof(Index));
```

ليه Redirect مش `return View()` ؟

ده نمط معروف اسمه **PRG — Post/Redirect/Get**. المشكلة اللي بيحلها:

من غير **PRG**:

1. المستخدم بيعت POST — السيرفر يحفظ ويعرض HTML مباشرة

2. المستخدم يعمل Refresh — المتصفح يسأل "تعيد الإرسال؟"

3. النتيجة: حفظ مكرر 🌟

مع **PRG**:

1. المستخدم بيعت POST — السيرفر يحفظ ويرد بـ Redirect

2. المتصفح يعمل GET للصفحة الجديدة

3. المستخدم يعمل Refresh — بيعيد الـ GET بس

4. النتيجة: مفيش تكرار ✓

`nameof(Index)` بترجع النص `"Index"` لكن بطريقة آمنة. لو غيّرت اسم الـ action، الكومبايلر هيقولك إن فيه خطأ. لو كتبت `"Index"` كنص عادي، الخطأ مكتش هيظهر غير وقت التشغيل.

5 و 6. Edit

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, [Bind("Id,Name,Description,CreatedAt")] Category ca
{
    if (id != category.Id) return NotFound();
    if (!ModelState.IsValid) return View(category);

    try
    {
        _context.Update(category);
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!await _context.Categories.AnyAsync(c => c.Id == id)) return NotFound();
        throw;
    }

    TempData["Success"] = "تم تعديل التصنيف";
    return RedirectToAction(nameof(Index));
}
```

```
if (id != category.Id) return NotFound();
```

فحص أمني. الـ `id` جاي من الراوت (`/Categories/Edit/5`)، و `category.Id` جاي من الفورم. لو مختلفين، يبقى فيه محاولة تلاعب.

```
catch (DbUpdateConcurrencyException)
```

سيناريو التزامن:

1. مستخدم أيفتح صفحة تعديل التصنيف رقم ١

2. مستخدم ب يفتح نفس الصفحة

3. مستخدم ب يحفظ ✓

4. مستخدم أ يحفظ — إيه اللي يحصل؟

الاستثناء ده بيتولد لما الصف اللي بتعدله اتغير أو اتحذف من تحتك.

الكود بيفرق بين حالتين: - اتحذف → رجّع 404 - اتغير → `throw` (أعد رمي الاستثناء عشان يظهر)

ملاحظة: الكشف الكامل لل التزامن يحتاج عمود `[Timestamp]` في الموديل. الكود ده يمكح حالة الحذف بشكل موثوق، والتغيير المتزامن يحتاج الإعداد الإضافي ده.

7. Delete

```
// GET - التأكيد
public async Task<IActionResult> Delete(int? id) { ... return View(category); }

// POST - التنفيذ الفعلي
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var category = await _context.Categories.FindAsync(id);
    if (category is not null)
    {
        _context.Categories.Remove(category);
        await _context.SaveChangesAsync();
    }
    TempData["Success"] = "تم حذف التصنيف";
    return RedirectToAction(nameof(Index));
}
```

— `[ActionName("Delete")]` — ليه؟

#C مبيسمحش بميثودين بنفس الاسم ونفس المعطيات:

```
public IActionResult Delete(int? id) // GET
public IActionResult Delete(int id) // POST — ❌ الكومبايلر هيرفض
```

الحل: سَمِّي الميثود اسم مختلف في #C، وقول لـ ASP.NET تعاملها كأنها `Delete`:

```
[HttpPost, ActionName("Delete")]
public async Task<IActionResult> DeleteConfirmed(int id)
```

دلوقتي `/Categories/Delete/5` بـ POST بتوصل للميثود دي، رغم إن اسمها الحقيقي `DeleteConfirmed`.

جرب بنفسك — الفصل 13

١. شيل `[ValidateAntiForgeryToken]` من `Create` وجرب تبعت POST من خارج الموقع (بـ Postman مثلاً). هينجح. رجّعها وجرب تاني — هيترفض بـ 400.

٢. شيل `[Bind]` من `Edit` وأضف حقل مخفي في الفورم:

```
<input type="hidden" name="CreatedAt" value="1990-01-01" />
```

هيتحفظ. ده الـ Over-posting بالظبط.

٣. غير `return RedirectToAction(nameof(Index))` لـ `return View(category)` بعد الحفظ. أضف تصنيف، ثم اعمل Refresh. المتصفح هيسألك تعيد الإرسال — وهيتضاف تاني. ده اللي PRG بيمنعه.

٤. أضف رسالة خطأ مخصصة:

```
if (await _context.Categories.AnyAsync(c => c.Name == category.Name))
{
    ModelState.AddModelError(nameof(Category.Name), "الاسم ده مستخدم قبل كده");
    return View(category);
}
```

حطها قبل `_context.Add(category)`. جرب تصنيف باسم موجود.

ProductsController.cs : الفصل 14

الفرق الأساسي: يستخدم `IProductService` بدل `AppDbContext`.

```
public class ProductsController : Controller
{
    private readonly IProductService _productService;
    private readonly AppDbContext _context; // فقط dropdown

    public ProductsController(IProductService productService, AppDbContext context)
    {
        _productService = productService;
        _context = context;
    }
}
```

ملاحظة صريحة: وجود الاثنين مع بعض هو حل وسط تعليمي بشأن تشوف النمطين جنب بعض. في مشروع حقيقي، كنت هعمل `IService` وأشير الـ `AppDbContext` من الكنترولر خالص.

الميثود المساعدة

```
private async Task PopulateCategoriesAsync(int? selectedId = null)
{
    var categories = await _context.Categories.AsNoTracking().OrderBy(c => c.Name).ToListAsync();
    ViewBag.Categories = new SelectList(categories, "Id", "Name", selectedId);
}
```

`SelectList` بتحوّل أي قائمة لـ `<option>` عناصر. معطياتها الأربعة:

المعطى	القيمة	الوظيفة
المصدر	categories	القائمة
dataValueField	"Id"	اللي هيتخزن في value
dataTextField	"Name"	اللي هيتظهر للمستخدم
selectedValue	selectedId	اللي هيبقى محدد مسبقاً

النتيجة في HTML:

```
<option value="1">معالجات</option>
<option value="2" selected>كروت شاشة</option>
```

ليه **private** ؟ لأنها مش action — مجرد مساعدة داخلية. لو خليتها **public** هتبقى قابلة للوصول كـ `/Products/PopulateCategoriesAsync` وده مش مطلوب.

النقطة الحرجة: إعادة التعبئة عند الخطأ

```
[HttpPost]
public async Task<IActionResult> Create([Bind(...)] Product product)
{
    if (!ModelState.IsValid)
    {
        await PopulateCategoriesAsync(product.CategoryId); // لازم ←
        return View(product);
    }

    await _productService.AddAsync(product);
    return RedirectToAction(nameof(Index));
}
```

ليه بننادي **PopulateCategoriesAsync** ثاني ؟

ViewBag عمره الطلب الواحد بس. الـ POST ده طلب جديد تماماً — الـ **ViewBag.Categories** اللي عيّناها في الـ GET راحت خلاص.

لو نسيت السطر ده: المستخدم هيدخل بيانات غلط، الصفحة هترجع، والـ **dropdown** هيبقى فاضي. وده من أشهر الأخطاء اللي المبتدئين بيوقعوا فيها.

١. شيل `await PopulateCategoriesAsync(product.CategoryId);` من داخل `if (!ModelState.IsValid)`. حاول تضيف منتج من غير اسم. الفورم هيرجع وال dropdown فاضي. ده الخطأ اللي اتكلمنا عنه.

٢. أضف action للبحث:

```
public async Task<IActionResult> Search(string q)
{
    var all = await _productService.GetAllAsync();
    var results = string.IsNullOrEmpty(q)
        ? all
        : all.Where(p => p.Name.Contains(q, StringComparison.OrdinalIgnoreCase));

    ViewBag.Query = q;
    return View("Index", results);
}
```

لاحظ `return View("Index", results)` — بنعيد استخدام نفس ال View باسم صريح.

سؤال: الكود ده بيفلتر في الذاكرة. إزاي تخليه يفلتر في الداتابيز؟ (رجع للفصل ١١، تمرين ١)

٣. حوّل الكنترولر يستخدم `ICategoryService` بدل `AppDbContext` وشيل ال Context خالص.

الفصل 15: ال Views ومحرك Razor

إيه هو Razor؟

Razor هو محرك قوالب يخلط HTML مع #C. الرمز @ هو نقطة التحوّل بين الاثنين.

```
<h1>@Model.Name</h1>                                <!-- طباعة قيمة -->
@if (Model.IsAvailable) { <span>متاح</span> }           <!-- شرط -->
@foreach (var p in Model) { <li>@p.Name</li> }           <!-- تكرار -->
@{ var total = Model.Price * Model.Stock; }           <!-- بلوك كود -->
```

! الحماية التلقائية من XSS

Razor يرمز أي مخرجات تلقائياً. لو منتج اسمه:

```
<script>alert('اخترقتك')</script>
```

@item.Name هيطبعه كنص عادي مرئي، مش هينفذ كسكربت. ال HTML المتولد:

```
&lt;script&gt;alert('اخترقتك')&lt;/script&gt;
```

دي حماية مجانية من XSS (Cross-Site Scripting) — أشهر ثغرة في تطبيقات الويب.

الاستثناء الخطير: @Html.Raw(value) بتطبع من غير ترميز. متستخدمهاش أبداً مع أي بيانات جاية من المستخدم.

_ViewImports.cshtml

```
@using ShopApp
@using ShopApp.Models
@using ShopApp.Services
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

يُطبق على كل ال Views. عشان كده بنكتب `@model Product` بدل `@model ShopApp.Models.Product`.

السطر الأخير بيغفل ال Tag Helpers — من غيره `asp-for` و `asp-action` هيقوا مجرد نصوص عادية في ال HTML.

_ViewStart.cshtml

```
@{
    Layout = "_Layout";
}
```

بيحدد القالب الافتراضي لكل ال Views.

_Layout.cshtml — القالب المشترك

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
    <title>@ViewData["Title"] - ShopApp</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.rtl.min.css" />
</head>
<body>
    <nav>...</nav>

    <div class="container">
        @if (TempData["Success"] is string msg) { <div class="alert alert-success">@msg</div> }
        <main>@RenderBody()</main>
    </div>

    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

— الاتنين مطلوبين للعربي. الأول بيقلب اتجاه النص، والثاني بيقلب تخطيط Bootstrap `bootstrap.rtl.min.css` و `dir="rtl"` كله (الأعمدة، الهوامش، المحاذاة).

@RenderBody() — هنا بيتحط محتوى ال View الحالية.

معناها **@await RenderSectionAsync("Scripts", required: false)** — نقطة إدراج للسكريبتات. **required: false** يعنيها إن ال View مش ملزمة تحدد القسم ده.

وال View بتملاه كده:

```
@section Scripts {  
    @await Html.PartialAsync("_ValidationScriptsPartial")  
}
```

ليه السكريبتات في الآخر؟ عشان الصفحة تظهر بسرعة. المتصفح بيرسم ال HTML الأول، وبعدين يحمل الجافاسكريبت.

ال Tag Helpers — التفصيل

ال Tag Helper بيخلي ال HTML يفضل HTML لكن بقوة السيرفر.

asp-for — الربط بالموديل

```
<input asp-for="Name" class="form-control" />
```

بيتولد:

```
<input class="form-control"  
  type="text"  
  id="Name"  
  name="Name"  
  value="القيمة الحالية"  
  data-val="true"  
  data-val-required="اسم التصنيف مطلوب"  
  data-val-length-max="60" />
```

لاحظ إنه استنتج ٦ حاجات لوحده:

المستنتج	من فين
<code>type="text"</code>	نوع الخاصية <code>string</code>
<code>name</code> و <code>id</code>	اسم الخاصية
<code>value</code>	قيمة الموديل الحالية
<code>data-val-*</code>	ال Data Annotations

ولو الخاصية `bool` هيبقى `type="checkbox"` ، ولو `DateTime` هيبقى `type="date"` .

`data-val-*` دي هي التحقق من جهة العميل — jQuery Validation بيقراها ويمنع الإرسال قبل ما يوصل السيرفر أصلاً.

`asp-action` و `asp-controller`

```
<a asp-controller="Products" asp-action="Edit" asp-route-id="@item.Id">تعديل</a>
```

بيتولد: `<a href="/Products/Edit/5">تعديل</a>`

ليه أحسن من كتابة الرابط يدوي؟ لأنه بيستخدم جدول الراوتنج. لو غيّرت قالب الراوت، كل الروابط بتتحدّث تلقائياً. لو كتبت `href="/Products/Edit/5"` يدوي، كنت هتفضل تدوّر عليها كلها.

`asp-route-<اسم>` بتضيف أي معطى للراوت. `asp-route-id="5"` → `/Products/Edit/5` .

`asp-items` — القوائم المنسدلة

```
<select asp-for="CategoryId" asp-items="@((ViewBag.Categories as SelectList))" class="form-select">
  <option value="">-- اختر التصنيف --</option>
</select>
```

ليه `as SelectList` ؟ لأن `ViewBag` نوعه `dynamic` — الكومبايلر مش عارف نوعه. ال `as` بتحوّله للنوع المتوقع.

`asp-validation-for` و `asp-validation-summary`

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div> <!-- أخطاء عامة -->
<span asp-validation-for="Name" class="text-danger"></span> <!-- خطأ حقل معيّن -->
```

قيم `asp-validation-summary` :

القيمة	بتعرض
ModelOnly	الأخطاء العامة بس (مش المرتبطة بحقل)
All	كل الأخطاء
None	مفیش

بنستخدم ModelOnly عشان أخطاء الحقول بتظهر جنب حقولها أصلاً — استخدام All كان هيكورها مرتين.

asp-append-version — كسر الكاش

```
<link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
```

بيتولد: `<link href="/css/site.css?v=Kl_dqr9NVtnMdsM2MUg4qthUnWZm5T1fCEimBPWDNgM" />`

القيمة دي بصمة (Hash) لمحتوى الملف. أول ما تعدّل الملف، البصمة تتغير، والمتصفح يعتبره ملف جديد ويحمّله. من غيرها ممكن المستخدم يفضل شايف CSS قديم من الكاش لأيام.

ال Views الخمسة — الأنماط

Index.cshtml — القائمة

```
@model IEnumerable<Product>
```

الموديل مجموعة — عشان كده @foreach .

```
@if (!Model.Any())
{
    <div class="alert alert-info">لا توجد منتجات بعد</div>
}
```

حالة الفراغ (Empty State) — دائماً اتعامل معاها. جدول فاضي بأعمدة بس تجربة سيئة.

Create.cshtml و Edit.cshtml

الفرق الوحيد المهم:

```
<input type="hidden" asp-for="Id" />
```

في **Edit** بس. من غيرها السيرفر مش هيعرف بتعدل مين.

في **Categories/Edit.cshtml** فيه حقل مخفي ثاني:

```
<input type="hidden" asp-for="CreatedAt" />
```

ليه؟ لأن **CreatedAt** مش في الفورم، فلو مبعتهاش هترجع **default** (سنة 0001) وتتحفظ كده، وتاريخ الإنشاء الأصلي يضيع.

ده بالظبط أحد الأسباب اللي بتخلي **ViewModels** حل أنصف — بدل ما تشيل حقول مخفية، بتقرا الكيان الأصلي من الداتابيز وتحدّث الحقول المسموحة بس.

Delete.cshtml

```
@if (Model.Products.Any())
{
    <div class="alert alert-danger">
        (Cascade Delete) منتج <strong>@Model.Products.Count</strong> تحذير: حذف هذا التصنيف سيحذف معه
    </div>
}
```

تحذير مشروط. المستخدم لازم يعرف عواقب فعله قبل ما ينقّذه. ده مبدأ أساسي في تصميم الواجهات.

جرب بنفسك — الفصل 15

١. أضف منتج اسمه:

```
<b>غامق</b>
```

افتح **/Products**. هتلاقيه ظهر كنص عادي مش غامق — دي حماية Razor التلقائية.

٢. غير **@item.Name** لـ **@Html.Raw(item.Name)** وشوف الفرق. رجّعها فوراً — ده الباب اللي بتدخل منه ثغرات XSS.

٣. شيل **<input type="hidden" asp-for="CreatedAt" />** من **Categories/Edit.cshtml**، عدّل تصنيف، وشوف تاريخ الإنشاء في صفحة التفاصيل. هيبقى **0001-01-01**.

٤. شيل `@section Scripts` من `Create.cshtml` وحاول تحفظ فورم ناقص. الرسالة هتظهر بس بعد ما الصفحة تروح السيرفر وترجع، مش فوراً. ده الفرق بين التحقق من جهة العميل والسيرفر.

٥. أضف Partial View جديدة. اعمل `Views/Shared/_ProductCard.cshtml` :

```
@model Product
<div class="card mb-2">
  <div class="card-body">
    <h5 class="card-title">@Model.Name</h5>
    <p class="card-text">@Model.Price.ToString("N2") - @Model.Category?.Name</p>
  </div>
</div>
```

واستخدما في `Index.cshtml` :

```
@foreach (var item in Model)
{
  <partial name="_ProductCard" model="item" />
}
```

الفصل 16: Migrations

المشكلة التي بتحلها

عندك موديلات في الكود، وجداول في الداتابيز. لما تضيف خاصية جديدة للموديل، الجدول لازم يتغير كمان.

الطرق الخاطئة: - تعدّل الجدول يدوي بـ SQL → زملأوك مش هيعرفوا يعملوا نفس التعديل - تمسح الداتابيز وتعملها من جديد → في الإنتاج ده يعني ضياع كل البيانات

Migrations هي الحل: كل تغيير بيتسجل كملف #C قابل للتطبيق والتراجع، وبيتحفظ في Git مع الكود.

محتوى ملف ال Migration

```
public partial class InitialCreate : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Categories",
            columns: table => new
            {
                Id = table.Column<int>(nullable: false)
                    .Annotation("Sqlite:Autoincrement", true),
                Name = table.Column<string>(maxLength: 60, nullable: false),
                // ...
            },
            constraints: table => { table.PrimaryKey("PK_Categories", x => x.Id); });

        // ... باقي الجداول والبيانات المبدئية ...
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(name: "Products");
        migrationBuilder.DropTable(name: "Categories");
    }
}
```

الميثود	متى تنفذ
Up()	عند database update — بتطبق التغيير
Down()	عند التراجع — بتلغي التغيير

لاحظ الترتيب في **Down**: يمحذف **Products** قبل **Categories**. لازم تحذف الجدول التابع الأول عشان قيد المفتاح الأجنبي.

الملفات الثلاثة

الملف	الدور
InitialCreate.cs< التاريخ>	أوامر الـ Up والـ Down
InitialCreate.Designer.cs< التاريخ>	لقطة كاملة من النموذج وقت إنشاء الـ Migration
AppDbContextModelSnapshot.cs	اللقطة الحالية — بتتحديث مع كل Migration

إزاي EF بيعرف إيه اللي اتغير؟ بيقارن الموديلات الحالية بـ **AppDbContextModelSnapshot.cs** — مش بالداتا بيز نفسها. الفرق بينهم هو الـ Migration الجديدة.

⚠ متعدلش **AppDbContextModelSnapshot.cs** يدوي أبداً. لو اتلخبط، EF هيوكد Migrations خاطئة.

جدول __EFMigrationsHistory

EF بيعمل جدول خاص في الداتا بيز:

```
CREATE TABLE "__EFMigrationsHistory" (
  "MigrationId" TEXT NOT NULL PRIMARY KEY,
  "ProductVersion" TEXT NOT NULL
);
```

فيه أسماء الـ Migrations المطبقة. وقت database update ، 1: EF. يقرأ الجدول ده 2. يقارنه بملفات فولدر **Migrations** 3. يطبق الناقص بس

ده اللي شفته في الترمينال أول تشغيل:

```
SELECT "MigrationId", "ProductVersion" FROM "__EFMigrationsHistory" ORDER BY "MigrationId";
Applying migration '20260819084454_InitialCreate'.
```

الأوامر الأساسية

```
# جديدة migration إنشاء
dotnet ef migrations add AddProductDescription

# المعلقة migrations تطبيق كل ال
dotnet ef database update

# (لو لسه ما اتطبقتش) migration التراجع عن آخر
dotnet ef migrations remove

# معينة migration الرجوع لـ
dotnet ef database update InitialCreate

# التراجع عن كل حاجة
dotnet ef database update 0

# عرض القائمة
dotnet ef migrations list

# بدل التطبيق المباشر (للإنتاج) SQL توليد سكريبت
dotnet ef migrations script

# سكريبت آمن يشتغل مهما كانت حالة الداتايز
dotnet ef migrations script --idempotent
```

قواعد ذهبية

١. سمي ال Migration باسم وصفي. `AddProductDescription` أحسن ألف مرة من `Migration2`. بعد سنة هتبص على التاريخ وتعرف حصل إيه.
٢. راجع الملف المتولد قبل ما تطبقه. EF ذكي بس مش معصوم. خصوصاً في تغييرات زي تغيير نوع عمود.
٣. متعدلش Migration اتطبقت بالفعل. اعمل واحدة جديدة تصحّح. لو عدلت المطبقة، مين طبقها قبلك هيبقى عنده حالة مختلفة عنك.
٤. في الإنتاج، استخدم السكريبتات.

```
dotnet ef migrations script --idempotent -o deploy.sql
```

راجع ال SQL بعينك قبل ما تشغله على بيانات حقيقية.

جرب بنفسك — الفصل 16

١. أضف خاصية للمنتج:

```
[StringLength(500)]
[Display(Name = "الوصف")]
public string? Description { get; set; }
```

اعمل Migration واقرا الملف:

```
dotnet ef migrations add AddProductDescription
```

هتلاقي **AddColumn** واحد بس — مش إعادة إنشاء الجدول.

٢. جرب التراجع:

```
dotnet ef migrations remove
```

هتلاقي الملفات اتمسحت وال Snapshot رجع لحالته. (ده بيشتغل لأنك ما طبقتهاش لسه).

٣. طبّقها، بعدين جرب تتراجع:

```
dotnet ef migrations add AddProductDescription
dotnet ef database update
dotnet ef database update InitialCreate      # السابقة migration رجوع للـ
dotnet ef migrations remove                  # دلوقتي ينفع تشيلها
```

٤. ولّد سكربت SQL وافتحه:

```
dotnet ef migrations script --idempotent -o migration.sql
```

اقراه. هتلاقي كل أمر مغلف في فحص **IF NOT EXISTS**. عشان كده اسمه idempotent — ينفع تشغله مليون مرة والنتيجة واحدة.

٥. امسح **shopapp.db** وشغّل **dotnet ef database update**. هتلاقي الداتا بيز اتبنت من الصفر بكل الـ Migrations بالترتيب. ده معنى إن الـ Migrations هي "تاريخ الداتا بيز".

الفصل 17: ملخص الأمان

الحماية	ضد إيه	فين في الكود
<code>[ValidateAntiForgeryToken]</code>	CSRF	كل POST action
<code>[Bind("...")]</code>	Over-posting	معطيات الـ Create والـ Edit
<code>ModelState.IsValid</code>	بيانات غير صالحة	بداية كل POST action
ترميز Razor التلقائي	XSS	كل @ في الـ Views
استعلامات EF المعاملية	SQL Injection	تلقائي في كل استعلام
<code>wwwroot</code> فقط مكشوف	كشف الملفات	<code>app.UseStaticFiles()</code>
صفحة خطأ عامة في الإنتاج	تسريب معلومات	<code>app.UseExceptionHandler</code>
<code>UseHsts</code> + <code>UseHttpsRedirection</code>	التنصت	الـ Pipeline

نقطة مهمة: SQL Injection

EF Core يستخدم معطيات (Parameters) تلقائياً. حتى لو المستخدم كتب:

```
' ; DROP TABLE Products; --
```

الاستعلام المتولد يبقى:

```
SELECT * FROM Products WHERE Name = @p0
-- @p0 = '' ; DROP TABLE Products; --
```

النص بيتعامل ك قيمة مش ك كود. آمن تماماً.

⚠️ الاستثناء الخطير: لو استخدمت SQL خام مع دمج نصوص:

// **✗** خطر شديد

```
_context.Products.FromSqlRaw($"SELECT * FROM Products WHERE Name = '{userInput}'");
```

// **✓** آمن

```
_context.Products.FromSqlInterpolated($"SELECT * FROM Products WHERE Name = {userInput}");
```

الفرق إن **FromSqlInterpolated** بتحوّل المتغيرات لمعطيات تلقائياً.

الفصل 18: أخطاء شائعة وحلولها

`Invalid column name 'X'` أو `no such column: X`

السبب: عدلت الموديل ونسيت ال Migration. الحل:

```
dotnet ef migrations add AddX
dotnet ef database update
```

`product.Category.Name` عند `NullReferenceException`

السبب: نسيت `.Include(p => p.Category)`. الحل: ضيف ال Include، أو استخدم `?.` في العرض.

`InvalidOperationException: The view 'Index' was not found`

السبب: ال View مش في المكان الصح. الحل: تأكد إن المسار

`<Action> اسم ال</> Controller اسم الكنترولر بدون كلمة Views/`

ال dropdown يرفض بعد خطأ في الفورم

السبب: نسيت تعيد تعبئة `ViewBag` في مسار الخطأ. الحل: نادِ `PopulateCategoriesAsync()` جَوّه `.if (!ModelState.IsValid)`

`Cannot consume scoped service from singleton`

السبب: سجّلت خدمة بتستخدم `DbContext` كـ `Singleton`. الحل: غيّرّها لـ `AddScoped`.

`dotnet ef` مش متعرّف عليه

الحل:

```
dotnet tool install --global dotnet-ef
```

ولو لسه، تأكد إن `~/dotnet/tools` في ال PATH.

Unable to create an object of type 'AppDbContext'

السبب: `dotnet ef` مش لاقى إزاي يعمل الـ `Context`. **الحل:** تأكد إنك في فولدر المشروع الصح، وإن `Program.cs` فيه `AddDbContext`.

التعديلات مش بتت حفظ

السبب: نسيت `SaveChangesAsync()` ، أو الكيان متجاب بـ `AsNoTracking()`. **الحل:** شيل `AsNoTracking` من الاستعلامات اللي هتعدّل عليها.

The instance of entity type 'Product' cannot be tracked

السبب: حاولت تعمل `Update` لكيان بينما نسخة ثانية بنفس الـ `Id` متتبعّة بالفعل. **الحل:** استخدم `AsNoTracking` وقت القراءة، أو حدّث الكيان المتتبع نفسه بدل ما تعمل `Update`.

الفصل 19: خطواتك الجاية

المستوى الأول — كَمَل الأساسيات

1. ViewModels ← الأهم على الإطلاق

المشكلة الحالية: بنستخدم ال Entities مباشرة في ال Views. ده بيخلينا نحتاج `[Bind]` وحقول مخفية.
الحل:

```
public class ProductCreateViewModel
{
    [Required, StringLength(100)]
    public string Name { get; set; } = string.Empty;

    [Range(0.01, 1_000_000)]
    public decimal Price { get; set; }

    public int CategoryId { get; set; }
    public IEnumerable<SelectListItem> Categories { get; set; } = new List<SelectListItem>();
}
```

الفوائد: مفيش over-posting أصلاً، مفيش `ViewBag`، والفورم بيحمل بالظبط اللي محتاجه.

2. Pagination — لو عندك ١٠٠٠ منتج، `/Products` هتنهار.

3. Search & Filter — بحث بالاسم وفلترة بالتصنيف والسعر.

4. Sorting — ترتيب بالضغط على رأس العمود.

المستوى الثاني — بنية أنصف

5. Repository + Unit of Work — طبقة تجريد فوق EF.

6. AutoMapper — تحويل تلقائي بين Entity و ViewModel.

7. FluentValidation — تحقق أقوى وأنصف من ال Data Annotations.

8. Serilog — تسجيل احترافي في ملفات أو قواعد بيانات.

المستوى الثالث — مميزات حقيقية

9. ASP.NET Core Identity — تسجيل دخول وصلاحيات كاملة.

10. Web API — حوّل المشروع لـ API:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsApiController : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>> Get() =>
        Ok(await _productService.GetAllAsync());
}
```

وبعدها اربطه بواجهة React أو Flutter.

11. رفع الصور — صورة لكل منتج.

12. Unit Testing — استخدم xUnit مع Moq. الـ `IProductService` التي عملناه جاهز للاختبار من دلوقتي.

مسار مقترح للأسابيع الجاية

الأسبوع	الموضوع
الأول	ViewModels + Pagination + Search
الثاني	Identity — تسجيل دخول وصلاحيات
الثالث	رفع الصور + AutoMapper
الرابع	Web API + Unit Tests

المصدر	الرابط
توثيق Microsoft الرسمي	learn.microsoft.com/aspnet/core
توثيق EF Core	learn.microsoft.com/ef/core
قناة Nick Chapsas	يوتيوب — محتوى متقدم وعملي
قناة Milan Jovanović	يوتيوب — معمارية وأنماط تصميم

كلمة أخيرة

المشروع ده صغير عن قصد — عشان تقدر تمسك كل خيط فيه من أوله لآخره. الفكرة مش إنك تحفظ الكود، الفكرة إنك تكسره وتصلحه.

كل تمرين في الدليل ده كتبته عشان تجرّبه فعلاً، مش تقراه. لما تشيل `Include` وتشوف العمود بيفضى، أو تشيل `[ValidateAntiForgeryToken]` وتشوف الطلب بيعدي — المعلومة دي بتتثبت بشكل مختلف تماماً عن مجرد قراءتها.

اكسر المشروع. ده أسرع طريقة تتعلم بيها.